

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 411 — Past Questions & Materials
by Renance

Course: CSC 411
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 9
Levels: 400

This study pack was compiled and restructured by Renance for CSC 411 students at Federal University of Petroleum Resources, Effurun. It brings together 9 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 9 — CSC 411 POSSIBLE QUESTIONS 2025

Level 400 • First Semester • PastQuestion

Question 1(Compulsory)

Requirement Analysis and Software Design

In the context of software engineering at FUPRE:

- Define what is meant by Requirement Analysis and list two benefits of conducting a thorough requirement analysis process. (2 marks)
- Differentiate between Functional Requirements and Non-Functional Requirements, providing one example for each. (5 marks)
- Explain the purpose of the Software Requirement Specification (SRS) Document and list three key sections that should be included in an SRS document. (5 marks)
- Describe the Layered Architecture style in software design and provide one advantage and one disadvantage of using this style. (5 marks)
- If a project team is designing a system for an online banking application, suggest one use case that could be modeled in a Use-Case Diagram and describe how an Association Relationship would be represented between an actor and the use case. (5 marks)

Total 22 marks

Question 2 (Compulsory)

Software Development Methodologies

- Briefly describe the Waterfall Methodology in software development. (2 marks)
- In a project using the Spiral Model, the team identifies risks early in each iteration. Explain how the Spiral Model handles risk management and why this makes it suitable for complex projects. (6 marks)
- Compare the Agile Methodologies with Sequential Methodologies in terms of flexibility to changing requirements. Provide one advantage of Agile over Sequential methodologies. (4 marks)
- Describe the Scrum Framework in Agile, focusing on the role of sprints and daily scrums., (4 marks)
- Given a scenario where a project requires rapid delivery of

a functional software prototype with modular components, recommend a suitable methodology from the ones discussed (Waterfall, Spiral, RAD, Incremental, Agile) and justify your choice. (6 marks)

Total 22 marks

Question 3

Software Design Patterns and Architecture

- a. Define what is meant by a Design Pattern in software engineering. (2 marks)
- b. Describe the Client-Server Pattern and provide one use case where it would be applicable. (4 marks)
- c. Explain the Microservices Pattern, including one advantage and one drawback of using this pattern in a cloud-based application. (4 marks)
- d. In the context of Object-Oriented Design, explain the difference between Aggregation and Composition relationships in a UML Class Diagram, providing an example for each. (6 marks)
- e. For a real-time stock trading platform, suggest a suitable architectural pattern from the ones discussed (e.g., Event-Driven, Data-Centered, Layered) and justify your choice with respect to the system's requirements. (6 marks)

Total 22 marks

Question 4

Application Programming Interfaces (APIs) and Software Reuse

- a. Define what an API is and explain its role in software development. (4 marks)
- b. Differentiate between Private APIs and Public APIs, providing one use case for each type. (4 marks)
- c. Describe the Request-Response Communication process in APIs, highlighting the role of the API Client and API Server. (4 marks)
- d. Explain the concept of Code Reusability in software engineering and list two techniques for achieving it. (4 marks)
- e. Given a scenario where a developer is building a weather forecasting application, suggest one API that could be used (from the examples provided) and describe how it would be integrated into the application. (6 marks)

Total 22 marks

Question 5

Software Life Cycle Models and Reuse

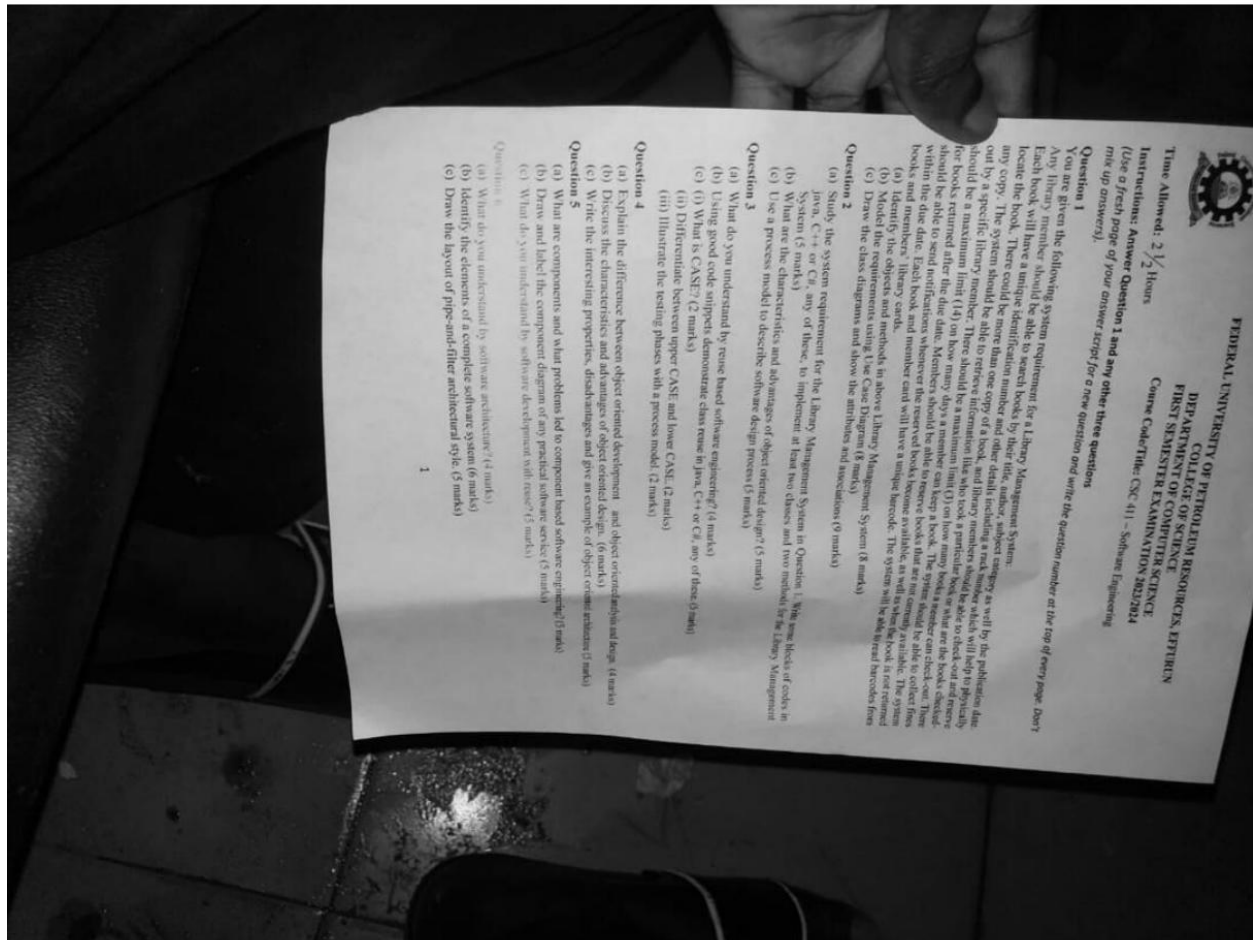
- a. Define the Incremental Model in software development. (2 marks)
- b. Describe the V-Model and explain how it differs from the Waterfall Model in terms of testing. (4 marks)
- c. In the context of Design for Reuse, explain what is meant by Component Reuse and provide one benefit of this approach. (4 marks)
- d. List and explain three benefits of Software Reuse as part of the development process. (6 marks)
- e. For a project developing a modular e-commerce platform, suggest a development methodology (from Waterfall, V-Model, RAD, Spiral, Incremental, or Agile) and explain how you would incorporate Software Reuse to enhance development efficiency. (6 marks)

Total 22 marks

Paper 2 of 9 — CSC 411 PQ

Level 400 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.



Paper 3 of 9 — CSC 411 past questions

Level 400 • First Semester • PastQuestion

Ate Gry, es Conk. act Spc

'aay op ere Sa oe

ae Bi atin resou-(p. Management - jf ha Ahad

to Vrowesble oulaas Shur. wt Onlpblolt tonal

hr i Vesoabt be toate _ Eile

AL ply Sin

Rie Badis

linked On Its arch,-

Paper 4 of 9 — CSC 411 Software Engineering Note 2 Course Handout

Level 400 • First Semester • Handout

SOFTWARE ENGINEERING NOTE 2 (CSC411)

Chapter 2: Software processes

A software process is a set of related activities that leads to the production of a software product. These activities may involve the development of software from scratch in a standard programming language like Java or C. However, business applications are not necessarily developed in this way. New business software is now often developed by extending and modifying existing systems or by configuring and integrating off-the-shelf software or system components. There are many different software processes but all must include four activities that are fundamental to software engineering:

1. Software specification: The functionality of the software and constraints on its operation must be defined.
2. Software design and implementation: The software to meet the specification must be produced.
3. Software validation: The software must be validated to ensure that it does what the customer wants.
4. Software evolution: The software must evolve to meet changing customer needs

In some form, these activities are part of all software processes. In practice, of course, they are complex activities in themselves and include sub-activities such as requirements validation, architectural design, unit testing, etc. There are also supporting process activities such as documentation and software configuration management. When we describe and discuss processes, we usually talk about the activities in these processes such as specifying a data model, designing a user interface, etc., and the ordering of these activities

PROCESS ACTIVITIES

The four basic process activities of specification, development, validation, and evolution are organized differently in different development processes. In the waterfall model, they are organized in sequence, whereas in incremental development they are interleaved. How these activities are carried out depends on the type of software, people, and organizational structures involved. In extreme programming, for example, specifications are written on cards. Tests are executable and developed before the program itself. Evolution may involve substantial system restructuring or refactoring

1. Software specification

1

Software specification or requirements engineering is the process of understanding

and defining what services are required from the system and identifying the constraints on the system's operation and development. Requirements engineering particularly critical stage of the software process as errors at this stage inevitably lead to later problems in the system design and implementation.

The requirements engineering process (Figure 2.4) aims to produce an agreed requirements document that specifies a system satisfying stakeholder requirements. Requirements are usually presented at two levels of detail. End-users and customers need a high-level statement of the requirements; system developers need a more detailed system specification.

There are four main activities in the requirements engineering process:

1. **Feasibility study:** An estimate is made of whether the identified user needs may be satisfied using current software and hardware technologies. The study considers whether the proposed system will be cost-effective from a business point of view and if it can be developed within existing budgetary constraints. A feasibility study should be relatively cheap and quick. The result should inform the decision of whether or not to go ahead with a more detailed analysis.
2. **Requirements elicitation and analysis:** This is the process of deriving the system requirements through observation of existing systems, discussions with potential users and procurers, task analysis, and so on. This may involve the development of one or more system models and prototypes. These help you understand the system to be specified.
3. **Requirements specification :** Requirements specification is the activity of translating the information gathered during the analysis activity into a document that defines a set of requirements. Two types of requirements may be included in this document. User requirements are abstract statements of the system requirements for the customer and end-user of the system; system requirements are a more detailed description of the functionality to be provided.
4. **Requirements validation:** This activity checks the requirements for realism, consistency, and completeness. During this process, errors in the requirements document are inevitably discovered. It must then be modified to correct these problems.

Feasibility

Study

2

Requirements

Elicitation and

Analysis

Requirements

Specification

Requirements

Validation

Feasibility

Report

System

Models

User and System

Requirements

Requirements

Document

Figure 2.4 The

requirements

engineering process

SOFTWARE DESIGN AND IMPLEMENTATION

The implementation stage of software development is the process of converting a system specification into an executable system. It always involves processes of software design and programming but, if an incremental approach to development is used, may also involve refinement of the software specification.

A software design is a description of the structure of the software to be implemented, the data models and structures used by the system, the interfaces between system components and, sometimes, the algorithms used. Designers do not arrive at a finished

design immediately but develop the design iteratively. They add formality and detail as

they develop their design with constant backtracking to correct earlier designs.

Figure 2.5 is an abstract model of this process showing the inputs to the design process, process activities, and the documents produced as outputs from this process

Platform

Information

Data

3

Description

Design Inputs

Design Activities

Design Outputs

Database Design

Figure 2.5 A general

model of the design

process

The diagram suggests that the stages of the design process are sequential. In fact, design process activities are interleaved. Feedback from one stage to another and consequent design rework is inevitable in all design processes.

Most software interfaces with other software systems. These include the operating system, database, middleware, and other application systems. These make up the 'software platform', the environment in which the software will execute. Information about this platform is an essential input to the design process, as designers must decide how best to integrate it with the software's environment. The requirements specification is a description of the functionality the software must provide and its performance and dependability requirements. If the system is to process existing data, then the description of that data may be included in the platform specification; otherwise, the data description must be an input to the design process so that the system data organization to be defined.

. Figure 2.5 shows four activities that may be part of the design process for information systems:

1. Architectural design, where you identify the overall structure of the system, the principal components (sometimes called sub-systems or modules), their relationships, and how they are distributed.

2. Interface design, where you define the interfaces between system components. This interface specification must be unambiguous. With a precise interface, a component can be used without other components having to know how it is implemented. Once interface specifications are agreed, the components can be designed and developed concurrently

3. Component design, where you take each system component and design how it will

- 4 operate. This may be a simple statement of the expected functionality to be implemented, with the specific design left to the programmer. Alternatively, it may be a list of changes to be made to a reusable component or a detailed design model.

The design model may be used to automatically generate an implementation.

4. Database design, where you design the system data structures and how these are

to be represented in a database. Again, the work here depends on whether an existing database is to be reused or a new database is to be created.

These activities lead to a set of design outputs, which are also shown in Figure 2.5.

The detail and representation of these vary considerably. For critical systems, detailed design documents setting out precise and accurate descriptions of the system must be produced. If a model-driven approach is used, these outputs may mostly be diagrams.

SOFTWARE VALIDATION

Software validation or, more generally, verification and validation (V&V) is intended to show that a system both conforms to its specification and that it meets the expectations of the system customer. Program testing, where the system is executed using simulated test data, is the principal validation technique. Validation may also involve checking processes, such as inspections and reviews, at each stage of the software process from user requirements definition to program development. Because of the predominance of testing, the majority of validation costs are incurred during and after implementation. Except for small programs, systems should not be tested as a single, monolithic unit.

Figure 2.6 shows a three-stage testing process in which system components are tested then the integrated system is tested and, finally, the system is tested with the customer's data.

Ideally, component defects are discovered early in the process, and interface problems are found when the system is integrated. However, as defects are discovered, the program must be debugged and this may require other stages in the testing process to be repeated. Errors in program components, say, may come to light during system testing. The process is therefore an iterative one with information being fed back from later stages to earlier parts of the process.

The stages in the testing process are:

5

1. Development testing The components making up the system are tested by the people developing the system. Each component is tested independently, without other system components. Components may be simple entities such as functions or object classes, or may be coherent groupings of these entities. Test automation tools, such as Unit that can re-run component tests when new versions of the component are created, are commonly used.
2. System testing System components are integrated to create a complete system. This process is concerned with finding errors that result from unanticipated interactions between components and component interface problems. It is also concerned with showing that the system meets its functional and non-functional requirements, and testing the emergent system properties. For large systems, this may be a multi-stage process where components are integrated to form subsystems that are individually tested before these sub-systems are themselves integrated to form the final system.

3. Acceptance testing This is the final stage in the testing process before the system is accepted for operational use. The system is tested with data supplied by the system customer rather than with simulated test data. Acceptance testing may reveal errors and omissions in the system requirements definition, because the real data exercise the system in different ways from the test data. Acceptance testing may also reveal requirements problems where the system's facilities do not really meet the user's needs or the system performance is unacceptable

Component

Testing

Acceptance

Testing

Figure 2.6 Stages

of testing

SOFTWARE EVOLUTION

The flexibility of software systems is one of the main reasons why more and more software is being incorporated in large, complex systems. Once a decision has been made to manufacture hardware, it is very expensive to make changes to the hardware design. However, changes can be made to software at any time during or after the system development. Even extensive changes are still much cheaper than corresponding changes to system hardware.

Historically, there has always been a split between the process of software development and the process of software evolution (software maintenance). People

6

think of software development as a creative activity in which a software system is developed from an initial concept through to a working system. However, they sometimes think of software maintenance as dull and uninteresting. Although the costs of maintenance are often several times the initial development costs, maintenance processes are sometimes considered to be less challenging than original software development.

This distinction between development and maintenance is increasingly irrelevant. Hardly any software systems are completely new systems and it makes much more sense to see development and maintenance as a continuum. Rather than two separate

processes, it is more realistic to think of software engineering as an evolutionary process (Figure 2.8) where software is continually changed over its lifetime in response to changing requirements and customer needs.

Define System

Requirements

Propose System

Changes

Modify
Systems
New
System
Existing
Systems
Figure 2.8 System
Evolution

2.1. Software Process Model

A software process model is a simplified representation of a software process. Each process model represents a process from a particular perspective, and thus provides only partial information about that process. For example, a process activity model shows the activities and their sequence but may not show the roles of the people involved in these activities. The process models covered here are:

1. The waterfall model: This takes the fundamental process activities of specification, development, validation, and evolution and represents them as separate process phases such as requirements specification, software design, implementation, testing, and so on
2. Incremental development: This approach interleaves the activities of specification, development, and validation. The system is developed as a series

7

of versions (increments), with each version adding functionality to the previous version.

3. Reuse-oriented software engineering: This approach is based on the existence of a significant number of reusable components. The system development process focuses on integrating these components into a system rather than developing them from scratch.

These models are not mutually exclusive and are often used together, especially for large systems development. For large systems, it makes sense to combine some of the best features of the waterfall and the incremental development models

2.2 The waterfall model

This model is illustrated in Figure 2.1. Because of the cascade from one phase to another, this model is known as the ‘waterfall model’ or software life cycle. The waterfall model is an example of a plan-driven process—in principle, you must plan and schedule all of the process activities before starting work on them. The principal stages of the waterfall model directly reflect the fundamental development activities:

1. Requirements analysis and definition: The system’s services, constraints, and goals are established by consultation with system users. They are then defined in detail and serve as a system specification.

2. System and software design :The systems design process allocates the requirements to either hardware or software systems by establishing an overall system architecture. Software design involves identifying and describing the fundamental software system abstractions and their relationships.
3. Implementation and unit testing: During this stage, the software design is realized as a set of programs or program units. Unit testing involves verifying that each unit meets its specification.
4. Integration and system testing: The individual program units or programs are integrated and tested as a complete system to ensure that the software requirements have been met. After testing, the software system is delivered to the customer.
5. Operation and maintenance: Normally (although not necessarily), this is the longest life cycle phase. The system is installed and put into practical use. Maintenance involves correcting errors which were not discovered in earlier stages of the life cycle, improving the implementation of system units and enhancing the system's services as new requirements are discovered.

8

2.3 Incremental development

Incremental development is based on the idea of developing an initial implementation, exposing this to user comment and evolving it through several versions until an adequate system has been developed (Figure 2.2). Specification, development, and validation activities are interleaved rather than separate, with rapid feedback across activities.

Incremental software development, which is a fundamental part of agile approaches, is better than a waterfall approach for most business, e-commerce, and personal systems. Incremental development reflects the way that we solve problems. We rarely work out a complete problem solution in advance but move toward a solution in a series of steps, backtracking when we realize that we have made a mistake. By developing the software incrementally, it is cheaper and easier to make changes in the software as it is being developed. Each increment or version of the system incorporates some of the functionality

that is needed by the customer. Generally, the early increments of the system include

the most important or most urgently required functionality. This means that the customer can evaluate the system at a relatively early stage in the development to see

if it delivers what is required. If not, then only the current increment has to be changed and, possibly, new functionality defined for later increments.

Incremental development has three important benefits, compared to the waterfall model:

1. The cost of accommodating changing customer requirements is reduced. The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
2. It is easier to get customer feedback on the development work that has been done. Customers can comment on demonstrations of the software and see how much has been implemented. Customers find it difficult to judge progress from software design documents.
3. More rapid delivery and deployment of useful software to the customer is possible, even if all of the functionality has not been included. Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

Concurrent

Activities

9

Validation Final

Version

Development

Intermediate

Versions

Specification Initial

Version

Outline

Description

Figure 2.2 Incremental

Development

Incremental development in some form is now the most common approach for the development of application systems.

From a management perspective, the incremental approach has two problems:

1. The process is not visible. Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.
2. System structure tends to degrade as new increments are added. Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly.

The problems of incremental development become particularly acute for large, complex, long-lifetime systems, where different teams develop different parts of the system. Large systems need a stable framework or architecture and the responsibilities of the different teams working on parts of the system need to be

clearly defined

with respect to that architecture. This has to be planned in advance rather than developed incrementally.

You can develop a system incrementally and expose it to customers for comment, without actually delivering it and deploying it in the customer's environment.

Incremental delivery and deployment means that the software is used in real, operational processes. This is not always possible as experimenting with new software can

disrupt normal business processes

10

2.4 Reuse-oriented software engineering

In the majority of software projects, there is some software reuse. This often happens

informally when people working on the project know of designs or code that are similar to what is required. They look for these, modify them as needed, and incorporate them into their system. This informal reuse takes place irrespective of the development process that is

used. However, in the 21st century, software development processes that focus on the

reuse of existing software have become widely used. Reuse-oriented approaches rely

on a large base of reusable software components and an integrating framework for the composition of these components. Sometimes, these components are systems in

their own right (COTS or commercial off-the-shelf systems) that may provide specific functionality such as word processing or a spreadsheet.

A general process model for reuse-based development is shown in Figure 2.3.

Although the initial requirements specification stage and the validation stage are comparable with other software processes, the intermediate stages in a reuse oriented process are different. These stages are:

1. Component analysis: Given the requirements specification, a search is made for

components to implement that specification. Usually, there is no exact match and the components that may be used only provide some of the functionality required.

2. Requirements modification: During this stage, the requirements are analyzed using

information about the components that have been discovered. They are then modified to reflect the available components. Where modifications are impossible, the

component analysis activity may be re-entered to search for alternative solutions.

3. System design with reuse: During this phase, the framework of the system is

designed or an existing framework is reused. The designers take into account the components that are reused and organize the framework to cater for this. Some new software may have to be designed if reusable components are not available.

4. Development and integration: Software that cannot be externally procured is developed, and the components and COTS systems are integrated to create the new system. System integration, in this model, may be part of the development process rather than a separate activity

11

Requirements

Specification

Component

Analysis

Development

and Integration

System Design

with Reuse

Requirements

Modification

System

Figure 2.3 Reuse-oriented Validation

software engineering

There are three types of software component that may be used in a reuse-oriented process:

1. Web services that are developed according to service standards and which are available for remote invocation.
2. Collections of objects that are developed as a package to be integrated with a component framework such as .NET or J2EE.
3. Stand-alone software systems that are configured for use in a particular environment.

Reuse-oriented software engineering has the obvious advantage of reducing the amount of software to be developed and so reducing cost and risks. It usually also leads to faster delivery of the software. However, requirements compromises are inevitable and this may lead to a system that does not meet the real needs of users. Furthermore, some control over the system evolution is lost as new versions of the reusable components are not under the control of the organization using them

ASSIGNT: Discuss the following (with examples):

12

1. Activity diagrams (2) Sequence diagrams (3) Class diagrams (4) use- case diagrams (5) UML diagrams (6) DFD

3.0 SOFTWARE ARCHITECTURE (ARCHITECTURAL DESIGN)

Architectural design is concerned with understanding how a system should be organized and designing the overall structure of that system. In the model of the software development process architectural design is the first stage in the software design process. It is the critical link between design and requirements engineering, as it identifies the main structural components in a system and the relationships between them. The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components.

Software architecture is important because it affects the performance, robustness, distributability, and maintainability of a system. In software architecture, individual components implement the functional system requirements. The nonfunctional requirements depend on the system architecture—the way in which these components are organized and communicate. In many systems, non-functional requirements are also influenced by individual components, but there is no doubt that the architecture of the system is the dominant influence.

There are three advantages of explicitly designing and documenting software architecture

1. Stakeholder communication: The architecture is a high-level presentation of the system that may be used as a focus for discussion by a range of different stakeholders.
2. System analysis: Making the system architecture explicit at an early stage in the system development requires some analysis. Architectural design decisions have a profound effect on whether or not the system can meet critical requirements such as performance, reliability, and maintainability.
3. Large-scale reuse: A model of a system architecture is a compact, manageable description of how a system is organized and how the components interoperate. The system architecture is often the same for systems with similar requirements and so can support large-scale software reuse.

13

Software architecture defines the fundamental structure, components, and relationships of a software system, acting as a blueprint to meet functional and non-functional requirements like scalability, security, and maintainability. It focuses on high-level design decisions that are difficult to change, guiding development to ensure long-term system health and performance.

Key Aspects of Software Architecture

□ Structure and Components: It defines the system's components (modules, services) and how they interact.

□ Non-functional Requirements (Quality Attributes):

Architecture ensures the system is modifiable, reliable, available, and secure.

□ Blueprint: It provides a, visual guide for development teams throughout the project lifecycle.

- Trade-off Analysis: Identifies and manages compromises, such as balancing performance against development speed.

Software Architecture defines fundamental organization of a system and more simply defines a structured solution. It determines how the various components of a software system are assembled, how they relate to one another, and how they communicate.

Essentially, it serves as a blueprint for the application and a foundation for the development team to build upon.

Software architecture defines a list of things which results in making many things easier in the software development process.

- System structure: The organization and arrangement of components.
- System behavior: The expected functionality and performance.
- Component relationships: How different parts of the system interact.
- Communication structure: The way components communicate with each other.
- Stakeholder balance: Meeting the needs and expectations of all stakeholders.
- Team structure: How the development team is organized and coordinated.
- Early design decisions: Making important choices early on to guide development.

Key Characteristics of Software Architecture

Software architecture is a multifaceted concept, and architects often categorize its characteristics based on various factors such as operation, requirements, and structure.

Here are some important characteristics to consider:

14

Operational Architecture Characteristics

- Availability: The system should be accessible when needed.
- Performance: The system should meet performance goals such as speed and responsiveness.
- Reliability: The system should work consistently without failure.
- Fault tolerance: The system should gracefully handle errors and failures.
- Scalability: The system should be able to handle increasing loads without performance degradation.

Structural Architecture Characteristics

- Configurability: The ability to configure the system according to needs.
- Extensibility: The system should be easily extendable to add new features.
- Supportability: The ease with which the system can be maintained and supported.
- Portability: The system should work across different environments.
- Maintainability: The system should be easy to update and fix over time.

Cross-Cutting Architecture Characteristics

- Accessibility: Ensuring the system is usable by a wide range of people, including those with disabilities.
- Security: Protecting the system from unauthorized access and data breaches.
- Usability: Ensuring the system is easy to use and intuitive for users.

- Privacy: Protecting users' sensitive information.
- Feasibility: The system should be realistic to develop within the constraints.

ASSIGNT 1. Discuss the S.O.L.I.D principle of software architecture

Advantages of Software Architecture

Good software architecture provides several advantages:

- Solid foundation: It lays the groundwork for a successful project, guiding development.
- Improved performance: A well-designed architecture can improve system efficiency.
- Reduced costs: Efficient development practices reduce costs over time.
- Scalability and flexibility: The system can adapt to future changes or demands.

Disadvantages of Software Architecture

While software architecture is essential, it does come with some challenges:

- Tooling and standardization: Obtaining the right tools and maintaining consistent standards can sometimes be a challenge.

15

- Uncertain predictions: It's not always possible to predict the success of a project based solely on its architecture.

Software architecture is fundamental to the successful development of any software application. It ensures that the system is scalable, maintainable, and adaptable to future changes. A good software architecture provides a blueprint that guides developers throughout the software development life cycle, helping to deliver high-quality, reliable, and efficient software products

KEYPOINTS

- A software architecture is a description of how a software system is organized.

Properties of a

system such as performance, security, and availability are influenced by the architecture used.

- Architectural design decisions include decisions on the type of application, the distribution of

the system, the architectural styles to be used, and the ways in which the architecture should be

documented and evaluated.

- Architectures may be documented from several different perspectives or views.

Possible views

include a conceptual view, a logical view, a process view, a development view, and a physical view.

- Architectural patterns are a means of reusing knowledge about generic system architectures.

They describe the architecture, explain when it may be used, and discuss its advantages and disadvantages.

■ Commonly used architectural patterns include Model-View-Controller, Layered Architecture,

Repository, Client-server, and Pipe and Filter.

16

Architecture Diagram

An architecture diagram is a visual framework used to capture every element of a piece of software during development. Architecture diagramming turns complex software into a simple visualization, allowing stakeholders to understand what the software does and how it does those things.

Software architecture Design Patterns

The architecture of a software system may be based on a particular architectural pattern or style. An architectural pattern is a description of a system organization such as a client-server organization or a layered architecture. Architectural patterns capture the essence of an architecture that has been used in different software systems. You should be aware of common patterns, where they can be used, and their strengths and weaknesses when making decisions about the architecture of a system. Software architecture patterns are high-level, reusable solutions to common problems in the overall structure of a software system. Software architecture patterns provide proven, reusable solutions to common problems at the system level. Examples include the Layered pattern (organizing by function), Microservices (small, independent services), and Event-Driven architecture (components communicating via events).

Here are examples of common software architecture design patterns:

Key Software Architecture Patterns

Pattern Description Key Use Cases

17

Layered Components are separated into Enterprise applications Architecture horizontal layers, each with a (CRM), e-commerce (N-tier) specific responsibility (e.g., platforms, and general presentation, business logic, data desktop applications. access).

Client-Server A system with a central server that Email systems, online provides resources or services to gaming, and remote file multiple client components that storage. request them.

Event-Driven Components communicate Real-time analytics, IoT

Architecture asynchronously through events, applications, and social (EDA) allowing for high scalability and media platforms. adaptability.

Microservices Structures an application as a E-commerce marketplaces collection of small, independent, (like Amazon), ride-sharing and self-contained services built applications, and streaming around specific business platforms (like Netflix). capabilities.

Microkernel Consists of a minimal core system Web browsers with (Plugin) and plug-in modules that provide extensions, text editors with extended functionality. plugins, and operating systems.

Pipe-Filter Data flows through a sequence of Data processing pipelines processing steps (filters) connected (ETL), compilers, and image by pipes. processing applications.

Broker A central component (broker) Messaging systems (like manages communication between Apache Kafka), distributed decoupled components in a applications, and web

18

distributed system. browsers.

System architectures are often modeled using simple block diagrams. Each box in the diagram represents a component. Boxes within boxes indicate that the component has been decomposed to sub-components. Arrows mean that data and or control signals are passed from component to component in the direction of the arrow.

1. Layered architecture

The layered architecture pattern is another way of achieving separation and independence. This pattern is shown in Figure 6.5. Here, the system functionality is organized into separate layers, and each layer only relies on the facilities and services offered by the layer immediately beneath it. This layered approach supports the incremental development of systems. As a layer is developed, some of the services provided by that layer may be made available to users. The architecture is also changeable and portable.

Name : Layered architecture

Description: Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent coreservices that are likely to be used throughout the system. See Figure 6.6.

Example : A layered model of a system for sharing copyright documents held in different libraries, as shown in Figure 6.7.

When used: Used when building new facilities on top of existing systems;

when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.

Advantages: Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.

Disadvantages : In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it.

Performance can be a problem because of multiple levels of minterpretation of a service request as it is processed at each layer

Figure 6.5 The layered architecture pattern

19

Figure 6.6 is an example of a layered architecture with four layers. The lowest layer includes system support software—typically database and operating system support. The next layer is the application layer that includes the components concerned with the application functionality and utility components that are used by other application components. The third layer is concerned with user interfacmanagement and providing user authentication and authorization, with the top layer providing user interface facilities. Of course, the number of layers is arbitrary. Any of the layers in Figure 6.6 could be split into two or more layers.

Figure 6.7 is an example of how this layered architecture pattern can be applied to a library system called LIBSYS, which allows controlled electronic access to copyright material from a group of university libraries. This has a five-layer architecture, with the bottom layer being the individual databases in each library.

2. CLIENT-SERVER ARCHITECTURE

The Client-server pattern is described in Figure 6.10

Name : Client-server

Description: In a client-server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.

Example: Figure 6.11 is an example of a film and video/DVD library organized as a client-server system.

When used : Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.

Advantages: The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing

service) can be available to all clients and does not need to be implemented by all services.

Disadvantages: Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system.

20

May be management problems if servers are owned by different organizations

Figure 6.10 The client-server pattern

A system that follows the client-server pattern is organized as a set of services and associated servers, and clients that access and use the services. The major components of this model are:

1. A set of servers that offer services to other components. Examples of servers include print servers that offer printing services, file servers that offer file management services, and a compile server, which offers programming language compilation services.
2. A set of clients that call on the services offered by servers. There will normally be several instances of a client program executing concurrently on different computers.
3. A network that allows the clients to access these services. Most client-server systems are implemented as distributed systems, connected using Internet protocols. Client-server architectures are usually thought of as distributed systems architectures but the logical model of independent services running on separate servers can be implemented on a single computer. Again, an important benefit is separation and independence. Services and servers can be changed without affecting other parts of the system. Clients may have to know the names of the available servers and the services that they provide. However, servers do not need to know the identity of clients or how many clients are accessing their services. Clients access the services provided by a server through remote procedure calls using a request-reply protocol such as the http protocol used in the WWW. Essentially, a client makes a request to a server and waits until it receives a reply.

Figure 6.11 is an example of a system that is based on the client-server model. This is a multi-user, web-based system for providing a film and photograph library. In this system, several servers manage and display the different types of media. Video frames need to be transmitted quickly and in synchrony but at relatively low resolution. They may be compressed in a store, so the video server can handle video compression and decompression in different formats. Still pictures, however, must be

21

maintained at a high resolution, so it is appropriate to maintain them on a separate server. The catalog must be able to deal with a variety of queries and provide links into the web information system that includes data about the film and video clips, and an e-commerce system that supports the sale of photographs, film, and video clips.

Figure 6.11 A client— server architecture for a film library

The client program is simply an integrated user interface, constructed using a web browser, to access these services. The most important advantage of the client-server model is that it is a distributed architecture. Effective use can be made of networked systems with many distributed processors. It is easy to add a new server and integrate it with the rest of the system or to upgrade servers transparently without affecting other parts of the system

22

Paper 5 of 9 — CSC411 -FUPRE- Software Engineering(2025) 3 Units

Level 400 • First Semester • Handout

CSC411 Software Engineering

Compiled by Dr. D. Oghorodi

Course Outline

1. Software Engineering Concepts:
 - 1.1 Definition of Software Engineering
 - 1.2 Types of Software Engineering
 - 1.3 Why is Software Engineering a branch of Engineering
 - 1.4 Roles and Functions of Software Engineer
2. Requirement Analysis
3. Software Design:
 - 3.1 Software Architecture Design
 - 3.2 Architectural Design Patterns
 - 3.3 Object Oriented Analysis & Design
 - 3.4 Design for Res-Use
4. Software Life Cycle Models
5. Using APIs
 - 2.1 API programming Class browsers and related tools
6. Component Based Computing
7. Software tools and Environment
 - 7.1 Design modelling tools,
 - 7.2 Testing Tools
 - 7.3 Tools for Integration mechanic

1. INTRODUCTION

Definition.

The term software engineering is the product of two words, software, and engineering.

Software refers to a set of instructions or programs, procedures, and data including associated documentation.

Engineering application of scientific principles to analyze, design, invent, code, build, and create a phenomenon to solve all sorts of problems

Software Engineering is the process of Analyzing, Designing, Coding, Testing and Deployment of software products that solve real-life problems by using well-defined scientific principles, methods, techniques, tools and procedures.

- Principles are scientific rules and laws that are generally accepted by scientists.
- Methods are approaches for doing something
- A technique is the know-how, procedure, routine, skill, style of carry out an activity
- Software tools are programs or applications that assist in the development, testing, and

maintenance of software.

- A procedure are steps and instructions for completing a task

Types of Software Engineering

1. Operational software engineering. It includes all decisions and tasks pertaining to how the software will perform within a computer system. This may include anything related to the software budget, the way teams and users will interact with the software and any potential risks such as those associated with defective and outdated software.
2. Transitional software engineering. This type of software engineering entails duties related to the software's adaptability and scalability when it's moved outside of its initial setting.
3. Software engineering maintenance. It entails activities connected to enhancing and debug current software to account for environmental changes, new technologies, bugs and risk factors that might have been disregarded during a previous development cycle. Over time, retirement takes over as maintenance of certain software is gradually reduced.

Software Engineering as a discipline of Engineering

The original use of the term is uncertain, but the first software engineering conference was held and sponsored by NATO in 1968. Software engineering is a new engineering discipline concerned with all aspects pertaining to software production. It encompasses concepts, principles, theories, techniques and tools that can be used for developing high-quality professional software.

Roles and responsibilities of Software engineer

- i. Design and management of software systems.
- ii. Testing and assessment of new software applications.
- iii. Code writing and evaluations.
- iv. Optimizing the speed and scalability of software.
- v. Consultation with customers, engineers, security experts and other parties involved.
- vi. Introducing new features to internal customers and stakeholders.

2. REQUIREMENT ANALYSIS

Requirements analysis is a process that involves identifying, analyzing, and managing the requirements for a Software Project. It is a process used to determine the needs and expectations of a new product. It involves frequent communication with the stakeholders and end-users of the product to define expectations, resolve conflicts, and document all the key requirements.

Software requirement analysis simply means complete study, analyzing, describing software requirements

Types of Requirements Analysis

All requirements can fit into four categories, whether they branch into various functions or serve a specific high-level function. The allocated requirements categories include:

1. Functional requirements
2. Operational requirements
3. Technical requirements

Functional requirements. Functional requirements define the specific behavior or functions, actions or the operations of a system. It is a statement of what a product (system, subsystem, system component, device or software program) must do.

It includes prescriptions of:

- Operations and workflows the product must perform
- Formats and validity of data to be input and output by the product
- User interface behavior
- Data integrity and data security requirements
- What the product must do to meet safety and other regulatory requirements
- How the system validates user access/authorization for use and modification of the system

Non-functional requirements are also called non-behavioral requirements. It state constraints on the design and construction of the product. They deal with issues like: Portability, Security, Maintainability, Reliability, Scalability, Performance, Reusability, and Flexibility

Operational requirements. The operational requirements focus on how the system will be operated by the users, including interfaces and interoperability with other systems. The requirements establish how well and under what conditions the system must perform. The operational requirements should answer questions like:

- Will be operating the system?
- When will the system be required to perform its intended function and for how long?
- What are the performance needs that must be met?
- How are the various system components to be used?
- How long will the system be in use by the user?

Technical requirements These, are the factors required to deliver a desired function or behavior from a system to satisfy a user's standards and needs

Requirement Analysis Process

This is done by communicating with users to gather requirement or eliciting requirements by

1. Conducting interviews
2. Observing the workplace
3. Holding focus groups or workplace
4. Gathering requirement list

Requirements Analysis Process Benefits

Some advantages of the requirements analysis:

1. A well-defined project's scope to fit the business process
2. More productive discussions among relevant stakeholders
3. Smooths project planning with fewer risks and technical issues
4. Resolve conflicts early in the software development process
5. A lower chance of miscommunication and rework
6. Better understanding among technical and non-technical stakeholders
7. The development team effort is data-driven with simple visual data flows

8. Team members have a shared understanding of business goals
9. Faster delivery within scheduled timelines to reach the target state
10. Teams enhance the product's performance with performance requirements
11. Your product satisfies the end user's expectations
12. The finished product has a higher chance of success

Software Requirement Specification (SRS) Document Format

SRS DOCUMENT FORMAT is a complete specification and description of requirements of the software. These requirements can be functional as well as non-functional depending upon the type of requirement.

Depending upon information gathered after interaction with users and other stakeholders, SRS is developed to satisfy customer's demand.

1. Introduction

- Purpose of this Document – The main aim of why this document is necessary and what's purpose of document is explained and described.
- Scope of this document – The overall working, main objective of document and what value it will provide to customer is described and explained.
- Overview – It simply gives the overall review of product.

2. General description

General functions of product which includes objective of user, a user characteristic, features, benefits, about why its importance is mentioned. It also describes features of user community.

3. Functional Requirements

Possible outcome of software system which includes effects due to operation of program is fully explained. All functional requirements which may include calculations, data processing, etc. are placed in a ranked order. Functional requirements specify the expected behavior of the system-which outputs should be produced from the given inputs. They describe the relationship between the input and output of the system. For each functional requirement, detailed description all the data inputs and their source, the units of measure, and the range of valid inputs must be specified.

4. Interface Requirements

Software interfaces which mean how software program communicates with each other or users either in form of any language, code, or message are fully described and explained. Examples can be shared memory, data streams, etc.

5. Performance Requirements

How a software system performs desired functions under specific condition is explained. It also explains required time, required memory, maximum error rate, etc. The performance requirements part of an SRS specifies the performance constraints on the software system. All the requirements relating to the performance characteristics of the system must be clearly specified. There are two types of performance requirements: static and dynamic. Static requirements are those that do not impose constraint on the execution

characteristics of the system. Dynamic requirements specify constraints on the execution behaviour of the system.

6. Design Constraints

Constraints which simply means limitation or restriction are specified and explained for design team. Examples may include use of a particular algorithm, hardware and software limitations, etc. There are a number of factors in the client's environment that may restrict the choices of a designer leading to design constraints such factors include standards that must be followed resource limits, operating environment, reliability and security requirements and policies that may have an impact on the design of the system. An SRS should identify and specify all such constraints.

7. Non-Functional Attributes

Non-functional attributes are explained that are required by software system for better performance. An example may include Security, Portability, Reliability, Reusability, Application compatibility, Data integrity, Scalability capacity, etc.

8. Preliminary Schedule and Budget

Initial version and budget of project plan are explained which include overall time duration required and overall cost required for development of project.

9. Appendices

Additional information like references from where information is gathered, definitions of some specific terms, acronyms, abbreviations, etc. are given and explained.

Features of an SRS

An SRS should have the following characteristics:

1. Correct. Should accurately reflect product functionality and specification at any point of time.
2. Unambiguous. There shouldn't be any confusion regarding interpretation of the requirements.
3. Complete. Should contain all the features the client requests.
4. Consistent. Same abbreviations and conventions must be followed throughout the document.
5. Ranked for importance and stability. Every requirement is important. But some are urgent and must be fulfilled before other requirements, and some can be delayed. Each requirement should be classified according to its importance and stability.
6. Verifiable. An SRS is verifiable only if every stated requirement can be verified. A requirement is verifiable if there's some method to quantifiably measure whether the final software meets that requirement.
7. Modifiable. An SRS must clearly identify each and every requirement in a systematic manner. If there are any changes, the specific requirements and the dependent ones can be modified accordingly without affecting the others.
8. Traceable. An SRS is traceable if the origin of each of its requirements is clear and if it

makes it easy to reference each requirement in future development.

3. SOFTWARE DESIGN

Software design is a process of transforming user requirements into some suitable form, which helps

the programmer in software coding and implementation.

Software design is a method that converts user requirements into a suitable form for the programmer to employ in software coding and implementation. It is concerned with converting the client's requirements as defined in the SRS (Software Requirement Specification) document into a form that can be easily implemented using a programming language.

Software design is the first step in moving the concentration from problem domain to solution domain

Objectives of Software Design

The following objectives describe what is software design in software engineering.

1. **Correctness:** A good design should be correct, which means that it should correctly implement all of the system's features.
2. **Efficiency:** A good software design should consider resource, time, and cost optimization parameters.
3. **Understandability:** A good design should be easy to grasp, which is why it should be modular, with all parts organized in layers.
4. **Completeness:** The design should include all components, such as data structures, modules, and external interfaces, among others.
5. **Maintainability:** A good software design should be flexible when the client issues a modification request.

Software Design Levels

There are three levels of software design.

1. **Architectural Design** - The architectural design is the highest abstract version of the system. It identifies the software as a system with many components interacting with each other. At this level, the designers get the idea of proposed solution domain.

Symbols of Architecture Diagram

<https://www.edrawmax.com/article/architecture-diagram-types.html>

2. **High-level Design**- The high-level design breaks the 'single entity-multiple component' concept of architectural design into less-abstracted view of sub-systems and modules and depicts their interaction with each other. High-level design focuses on how the system along with all of its different components of the system work together without the internal coding and implementation details.

3. **Low-Level Design**- Low-level designing is also known as object-level designing or micro-level or detailed designing. Low-Level Design, is where detailed system components and their interactions are specified, along with their implementation. It involves converting the high-level design into a more detailed blueprint, addressing

specific algorithms, data structures, and interfaces. LLD serves as a guide for developers during coding, which ensures the accurate and efficient implementation of the system's functionality.

3.1 Software Architectural Design

Software Architecture It defines how components of a software system are assembled, their relationship and communication between them.

The software needs an architectural design to represent the system works. IEEE defines architectural design as “the process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system.” The software that is built for computer-based systems can exhibit one of these many architectural styles.

Software architecture defines a list of things which results in making many things easier in the software development process.

- A software architecture defines structure of a system.
- A software architecture defines behavior of a system.
- A software architecture defines component relationship.
- A software architecture defines communication structure.
- A software architecture balances stakeholder's needs.
- A software architecture influences team structure.
- A software architecture focuses on significant elements.
- A software architecture captures early design decisions.

System Category Consists of:

- A set of components (eg: a database, computational modules) that will perform a function required by the system.
- The set of connectors will help in coordination, communication, and cooperation between the components.
- Conditions that define how components can be integrated to form the system.
- Semantic models that help the designer to understand the overall properties of the system.

The use of architectural styles is to establish a structure for all the components of the system.

Taxonomy of Architectural Styles

1. Data centered architectures:

- A data store resides at the center of the architecture and it is accessed frequently by the other components that update, add, delete, or modify the data present within the store.
- The client software accesses a central repository. Variations of this approach are used to transform the repository into a blackboard when data related to the client or data of interest for the client change the notifications to client software.

1. This data-centered architecture will promote integrability. This means that the existing

components can be changed and new client components can be added to the architecture without the permission or concern of other clients.

2. Data can be passed among clients using the blackboard mechanism.

Advantages of Data centered architecture:

1. Repository of data is independent of clients
2. Client work independent of each other
3. It may be simple to add additional clients.
4. Modification can be very easy

2. Data flow architectures:

In data flow architecture, the whole software system is seen as a series of transformations on consecutive pieces or set of input data, where data and operations are independent of each other. In this approach, the data enters into the system and then flows through the modules one at a time until they are assigned to some final destination (output or a data store).

The connections between the components or modules may be implemented as I/O stream, I/O buffers, piped, or other types of connections. The data can be flown in the graph topology with cycles, in a linear structure without cycles, or in a tree type structure.

The main objective of this approach is to achieve the qualities of reuse and modifiability.

It is suitable for applications that involve a well-defined series of independent data transformations or computations on orderly defined input and output such as compilers and business data processing applications. There are three types of execution sequences between modules:

- Batch sequential
- Pipe and filter or non-sequential pipeline mode
- Process control

Advantages of Data Flow architecture:

1. It encourages upkeep, repurposing, and modification.
2. With this design, concurrent execution is supported.

Disadvantage of Data Flow architecture:

- i. It frequently degenerates to batch sequential system
- ii. Data flow architecture does not allow applications that require greater user engagement.
- iii. It is not easy to coordinate two different but related streams

3. Call and Return architectures

It is used to create a program that is easy to scale and modify. Many sub-styles exist within this category. Two of them are explained below.

i. Remote procedure call architecture: This component is used to present in a main program or sub program architecture distributed among multiple computers on a network.

ii. Main program or Subprogram architectures: The main program structure decomposes into number of subprograms or function into a control hierarchy. Main

program contains number of subprograms that can invoke other components.

4. Object Oriented architecture

The structure and behavior of any software application can be represented through the use of multiple and

interoperable objects. Object-Oriented Architecture is a design paradigm based on the division of responsibilities for an application or system into individual reusable and self-sufficient objects where Objects connect, communicate, and collaborate through well-defined interfaces via message passing.

Characteristics of Object Oriented architecture:

- i. Object protect the system's integrity.
- ii. An object is unaware of the depiction of other items.

Advantage of Object Oriented architecture:

- i. It enables the designer to separate a challenge into a collection of autonomous objects.
- ii. Other objects are aware of the implementation details of the object, allowing changes to be made without having an impact on other objects.

5. Layered architecture

The layered architecture style is one of the most common architectural styles. The idea behind Layered Architecture is that modules or components with similar functionalities are organized into horizontal layers. As a result, each layer performs a specific role within the application.

The layered architecture style does not have a restriction on the number of layers that the application can have, as the purpose is to have layers that promote the concept of separation of concerns. The layered architecture style abstracts the view of the system as a whole while providing enough detail to understand the roles and responsibilities of individual layers and the relationship between them.

Example of this architectural style is OSI-ISO (Open Systems Interconnection-International Organization for Standardization) communication system.

What Makes A Good Software Architecture?

An efficient programming architecture has a set of quality attributes as mentioned below:

1. Functionality: Refers to the extent the software performs against its needed purposes.
2. Usability: Refers to the level the software can be used with ease and convenience.
3. Reliability: Refers to the capability of the product to provide intended functionality under given circumstances.
4. Supportability: Refers to the facility with which developers can transfer the software from one platform to another with minimal or no changes.
5. Performance: Refers to the approximation by considering resource utilization, processing speed, response time, productivity, and throughput.
6. Self-Reliance: refers to the capability of independent activities for optimal performance even if one is going through a downtime.

3.2 Software Design Patterns

A design pattern is a general repeatable solution to a commonly occurring problem in software design. A design pattern is not a finished design that can be transformed directly into code but a description or template for how to solve a problem that can be used in many different situations.

With Design Patterns as your guide, you will learn how these important patterns fit into the software development process, and how you can leverage them to solve your own design problems most efficiently.

Categories of Design patterns

There are 23 classic Design Patterns, although at least 26 Design Patterns have been discovered to date

The Gang of Four (GOF) Design Patterns refer to a set of 23 foundational software design patterns introduced by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides in their seminal book, "Design Patterns: Elements of Reusable Object-Oriented Software." Published in 1994, This book categorizes patterns into three types: creative, structural, and behavioral.

These Twenty-three of the design patterns are the most well-known and are fundamental, hence they are nicknamed "Gangs of Four" (GoF) design patterns because of these four authors.

The code examples of these design patterns can be found in

<https://refactoring.guru/design-patterns/prototype/cpp/example#example-0>

Creational patterns. In software engineering, creational design patterns are design patterns that deal with object creation mechanisms. This pattern can be further divided into class-creation patterns and object-creational patterns. While class-creation patterns use inheritance effectively in the instantiation process, object-creation patterns use delegation effectively to get the job done.

Structural design patterns These design patterns are all about Class and Object composition.

Structural class-creation patterns use inheritance to compose interfaces. Structural object-patterns define ways to compose objects to obtain new functionality.

Behavioral design patterns These design patterns are all about Class's objects communication.

Behavioral patterns are those patterns that are most specifically concerned with communication between objects.

Some Specific Software Architecture Design Patterns

There are 23 design patterns which allow designers to create more flexible, elegant, and ultimately reusable designs without having to rediscover the design solutions

1. **Layered Pattern:** It is one of the most common types of architecture in software engineering. Organizes software into horizontal layers, each responsible for distinct functionalities like presentation, business logic, and data storage. Enables modular development and maintenance, commonly used in web applications.

Use cases

- E-commerce Platform.
- Banking Application:
- Content Management System.

Drawbacks

- Communication overhead between layers can impact performance.
- Tight coupling risks if layer boundaries aren't well-defined.
- May become overly complex with numerous layers.

2. Client-Server Pattern Separates application into clients (user interfaces) and servers (data processing) to manage data sharing and user interactions. Ideal for distributed systems like web-based services.

Uses Cases

- Email System: Allows clients to send requests for retrieving or sending emails to a central server.
- Online Gaming: Clients interact with a central server for real-time game updates and multiplayer interactions.
- Remote File Storage: Clients access a server to store and retrieve files securely from a remote location.

Drawbacks

- Server scalability challenges during heavy traffic periods.
- Complex communication management between clients and server.
- Potential single point of failure if the server goes down.

3. Event-Driven Pattern Even-Emphasizes communication between components through asynchronous events, triggered by user actions or data changes. Used in real-time systems and graphical user interfaces.

Use Cases

- Social Media Platform: Users' actions like posting, liking, or commenting trigger event-driven updates across the platform.
- Stock Trading Platform: Rapid response to real-time market changes, executing buy/sell orders in reaction to market events.
- Smart Home System: Devices react to user inputs like turning lights on/off based on sensor-triggered events.

Drawbacks

- Debugging can be complex due to non-linear event flows.
- Event order and timing can introduce unexpected behavior.
- Overuse of events can lead to convoluted architectures.

4. Micro-Kernel Pattern Divides core functionality from optional features, allowing extensible applications through plugins. Suited for software requiring easy feature expansion.

Use Cases

- Text Editor with Plugins: Core functionality for text editing, with plugins adding specialized features like code highlighting or spell checking.
- Web Browser with Extensions: Core browser functionality complemented by extensions for ad-blocking or password management.
- Music Player with Skins: Core music playback capabilities extended by skins that change

the player's visual appearance.

Drawbacks

- Communication between core and plugins can introduce overhead.
- Plugins may have dependencies on specific core versions.
- Managing interactions between core and plugins can become complex.

5. Micro Service Pattern Structures applications as a collection of small, independently deployable services, enabling scalability and rapid development. Common in cloud-based

Use Cases

- E-commerce Marketplace: Different microservices handle user management, product catalog, payments, and order processing.
- Ride-Sharing Application: Separate services manage user authentication, ride requests, driver tracking, and payments.
- Streaming Platform: Microservices for content delivery, user profiles, recommendations, and billing.

Drawbacks

- Complexity in managing distributed architecture.
- Challenges in ensuring data consistency across services.
- Communication overhead between services can impact performance.

6. Broker Patterns : Introduces a central broker that handles communication between distributed components, enhancing decoupling and efficiency. Commonly used in messaging systems.

Use Cases

- Financial Market Data: Brokers distribute real-time stock market data to various clients for analysis and trading decisions.
- Message Queues: Brokers manage message distribution between multiple components, aiding asynchronous communication.
- Internet of Things (IoT) Hub: Broker facilitates communication between IoT devices and cloud services

Drawbacks

- Central broker becomes single point of failure.
- Message routing introduces potential latency.
- Broker's capacity may limit scalability.

7. Event-Bus Pattern Components communicate through an event bus by publishing and subscribing to events. Facilitates loose coupling and is prevalent in modular applications.

Use Cases

- Modular Video Game: Different game systems interact through events, such as player actions affecting the game world or triggering animations.
- E-commerce Checkout Process: Events signal each step of the checkout process, from adding items to the cart to confirming the order.

- Workflow Automation: Events drive the progression of tasks in a business process, like document approvals or task completion.

Drawbacks

- Debugging can be complex due to decentralized event propagation.
- Overuse of events can lead to convoluted interactions.
- Ensuring correct event order and managing subscriptions can be challenging.

8. Pipe-Filter Pattern Data flows through a series of filters arranged in a pipeline to achieve data transformation or processing. Common in data processing systems.

Use Cases

- Image Processing: Filters in a pipeline transform images step by step, applying effects like blurring or color adjustments.
- Data ETL (Extract, Transform, Load): Filters process and transform data as it flows through a pipeline, preparing it for analysis.
- Audio Signal Processing: Filters modify audio signals in sequence, such as noise reduction or equalization.

Drawbacks

- Overemphasis on filters can lead to rigid architecture.
- Managing filter order and interactions can become complex.
- Complex pipelines can be challenging to manage and troubleshoot.

9. Blackboard Pattern Specialized agents contribute to a shared knowledge repository (blackboard), collaborating to solve complex problems, commonly found in AI systems.

Use Cases

- Medical Diagnosis: Various agents contribute knowledge to a blackboard, collaborating to diagnose complex medical conditions.
- Scientific Data Analysis: Researchers share findings through a blackboard, combining data from different sources for insights.
- Natural Language Processing: Agents contribute linguistic knowledge to a blackboard, collaborating to understand and generate language.

10. Component-Based Pattern Breaks down software into reusable components with well-defined interfaces, enhancing code reusability and maintainability. Often used in GUI frameworks and SDKs.

Use Cases

- Graphic Design Software: Components handle tools like drawing, text editing, and filters, contributing to a comprehensive design suite.
- GUI Library: Reusable components provide buttons, text fields, and other UI elements for building user interfaces.
- Financial Software Suite: Different components manage tasks like accounting, payroll, and invoicing within a comprehensive suite.

Drawbacks

- Over-fragmentation can lead to challenges in managing dependencies.
- Determining appropriate component boundaries may require careful design.
- Interactions between components need to be carefully managed.

11. Service-Oriented Architecture (SOA) A style where applications are composed of services that communicate over a network. Each service is a self-contained unit with a well-defined interface, and they work together to provide higher-level functionality.

Use Cases

- Enterprise Systems: Large organizations use SOA to integrate various departments' systems, like HR, finance, and sales.
- E-commerce Integration: Services from different vendors can be combined to create a unified online shopping experience.
- Legacy System Integration: SOA enables integrating older systems with new ones without a full rewrite.

Drawbacks

- Complex to design and manage services.
- Overhead due to network communication.
- Service versioning can be challenging.

12. Monolithic Architecture An older approach where all components of an application are tightly integrated into a single codebase and are deployed together. While less common now, it's still seen in some legacy systems.

Use Cases

- Small to Medium Web Applications: Simplicity can be an advantage for projects with limited complexity.
- Rapid Prototyping: Quick development and deployment for initial versions of software.
- Legacy Systems: Existing monolithic applications that have been in use for years.

Drawbacks

- Limited scalability, as the entire application must be scaled.
- Difficulty in maintaining and updating due to tight coupling.
- Deployment of updates can be riskier.

13. Space-Based Architecture A distributed approach where data and processing are spread across multiple nodes in a space-like grid, often used for applications with high scalability requirements.

Use Cases

- High-Performance Computing: Space-based architecture efficiently distributes computational tasks across a cluster.
- Real-Time Analytics: Distributed processing of data streams for immediate insights.
- Multiplayer Online Games: Scalable architecture for handling massive numbers of concurrent players.

Drawbacks

- Complex to implement and manage.
- Distributed data management and synchronization challenges.
- Network latency can impact performance.

14. Peer-to-Peer Architecture Nodes in the network act both as clients and servers, sharing resources directly without a centralized server. Often used in decentralized file-sharing systems.

Use Cases

- Decentralized File Sharing: Users share files directly with each other without a central repository.
- Blockchain Networks: Distributed ledgers where each node maintains a copy of the entire blockchain.
- Collaborative Tools: Peer-to-peer architecture allows direct sharing of resources in collaborative applications.

Drawbacks

- Security concerns due to direct connections between nodes.
- Scalability challenges in very large networks.
- Lack of central control can lead to coordination issues.

15. Hybrid Architecture Combines multiple architectural patterns to address specific application requirements. For example, combining microservices with event-driven patterns.

Use Cases

- Complex Enterprise Systems: Hybrid architectures can balance the strengths of different patterns to meet diverse needs.
- Scalable Web Applications: Combining microservices with event-driven patterns to ensure responsiveness and modularity.
- Real-Time Analytics: Using a combination of event-driven and space-based patterns for efficient data processing.

Drawbacks

- Complexity in managing hybrid architectures.
- Integration challenges between different patterns.
- Requires careful design and planning to ensure cohesiveness.

Key Characteristics of Design Patterns

- Reusability: Patterns can be applied to different projects and problems, saving time and effort in solving similar issues.
- Standardization: They provide a shared language and understanding among developers, helping in communication and collaboration.
- Efficiency: By using these popular patterns, developers can avoid finding the solution to same recurring problems, which leads to faster development.
- Flexibility: Patterns are abstract solutions/templates that can be adapted to fit various scenarios and requirements.

Why Learn Design Patterns?

There are multiple reasons to learn design patterns:

1. Code that is simpler to comprehend, update, and expand is produced with the help of design patterns.
2. They offer solutions that have been tried and tested as well as best practices.
3. Learning this enables them to quickly and effectively address similar challenges in various projects.
4. Developers can produce reusable components that can be utilized in a variety of applications by implementing design patterns.
5. This reduces redundancy and saves development time.

Criticisms

The concept of design patterns has been criticized by some in the field of computer science in that:

- i. Targets the wrong problem
- ii. Lacks formal foundations
- iii. Leads to inefficient solutions
- iv. Does not differ significantly from other abstractions

3.3 Object Oriented Design

<https://www.geeksforgeeks.org/unified-modeling-language-uml-introduction/?ref=shm>

Unified Modeling Language (UML) is a standardized visual modeling language that is a versatile, flexible, and user-friendly method for visualizing a system's design. Software system artifacts can be specified, visualized, built, and documented with the use of UML. Therefore, UML diagrams are the building blocks for object-oriented design

Types of UML Diagrams

The UML diagrams are divided into two parts: Structural UML diagrams and Behavioural

Structural UML Diagrams

Structural UML diagrams, show visual representations that depict the static aspects of a system , including its classes, objects, components, and their relationships that provides a clear view of the system's architecture

4.1. Class Diagram

The most widely use UML diagram is the class diagram. It shows the system's classes, their methods and attributes. It also helps to identify relationship between different classes or objects.

Example.

Animal is a class. A duck is a type of Animal and so it is an object in Animal class.

A duck has attributes or states - color, name, breed and behaviors -swim, quack

A class diagram shows the following

- Classes name,
- their attributes,

- operations (or methods),
- and the relationships among objects.

Class Visibility

The +, - and # symbols denote the visibility of the attribute and operation.

- + denotes public attributes
- - denotes private attributes
- # denotes protected attributes or operations

Relationships between classes

A class may be involved in one or more relationships with other classes. A relationship can be one of the following types:

1. Inheritance (or Generalization): Here, SubClass1 and SubClass2 are derived from SuperClass. The relationship is displayed as a solid line with a hollow arrowhead that points from the child element to the parent element.

Therefore, the child elements (SubClass1 and SubClass2) inherited the attributes of superclass

Other styles

2. Association: Associations are relationships between classes in a UML Class Diagram. They are represented by a solid line between classes. Associations are typically named using a verb or verb phrase which reflects the real-world problem domain.

2.1 Simple Association

- A structural link between two peer classes.
- There is an association between Class1 and Class2

The figure below shows an example of simple association. There is an association that connects the <<control>> class Class1 and <<boundary>> class Class2.

The relationship is displayed as a solid line connecting the two classes.

2.2 Cardinality: Cardinality is expressed in terms of:

- one to one
- one to many
- many to many

3. Aggregation : A special type of association.

- It represents a "part of" relationship.
- Class2 is part of Class1.
- Many instances (denoted by the *) of Class2 can be associated with Class1.
- Objects of Class1 and Class2 have separate lifetimes.

In aggregation, relationship is displayed as a solid line with a unfilled diamond at the association end, which is connected to the class that represents the aggregate.

4. Composition

- A special type of aggregation where parts are destroyed when the whole is destroyed.

- Objects of Class2 live and die with Class1.
- Class2 cannot stand by itself.

The relationship is displayed as a solid line with a filled diamond at the association end, which is connected to the class that represents the whole or composite.

5. Dependency. An object of one class might use an object of another class in the code of a method. If the object is not stored in any field, then this is modeled as a dependency relationship.

- A special type of association.
- Exists between two classes if changes to the definition of a class may cause changes to the other class (but not the other way around).
- Class1 depends on Class2

The relationship is displayed as a dashed line with an open arrow.

The figure below shows another example of dependency.

6. Realization. Realization is a relationship between the blueprint class and the object containing its respective implementation level details. This object is said to realize the blueprint class. In other words, you can understand this as the relationship between the interface and the implementing class.

The Person and Corporation classes need to implement these methods, possibly in very different ways.

Example Class Diagram of Order System

m

Behavioural UML Diagrams

Use-case Diagram

In the Unified Modeling Language (UML), a use case diagram can summarize the details of your system's users (also known as actors) and their interactions with the system. That is, it shows how different users engage with various use cases, or specific functionalities, within the system. It provides a high-level view of the system's functionality by illustrating the various ways users can interact with it.

Elements of a Use-Case Diagram

1. Actors: Actors are external entities that interact with the system. Actors include users, other systems, or hardware devices. The actors initiate use cases and receive the outcomes. Proper identification and understanding of actors are crucial for accurately modeling system behavior.
2. Use Cases: Use cases represent specific things your system can do. In the online shopping system, examples of use cases could be "Place Order," "Track Delivery," or "Update Product Information". Use cases are represented by ovals shape symbols.
3. System Boundary: The system boundary is a visual representation of the scope or limits of the system you are modeling. It defines what is inside the system and what is outside. The boundary helps to establish a clear distinction between the elements that are

part of the system and those that are external to it. The system boundary is typically represented by a rectangular box that surrounds all the use cases of the system

The purpose of system boundary is to clearly outlines the boundaries of the system, indicating which components are internal to the system and which are external actors or entities interacting with the system.

Types of relationships their usage Use-case Diagram.

1. Association Relationship: The Association Relationship represents a communication or interaction between an actor and a use case. It is depicted by a line connecting the actor to the use case. This relationship signifies that the actor is involved in the functionality described by the use case.

Example: Online Banking System

- Actor: Customer
- Use Case: Transfer Funds
- Association: A line connecting the “Customer” actor to the “Transfer Funds” use case, indicating the customer’s involvement in the funds transfer process.

2. Include Relationship: The Include Relationship indicates that a use case includes the functionality of another use case. It is denoted by a dashed arrow pointing from the including use case to the included use case.

Example: Social Media Posting

- Use Cases: Compose Post, Add Image
- Include Relationship: The “Compose Post” use case includes the functionality of “Add Image.” Therefore, composing a post includes the action of adding an image.

3. Extend Relationship: The Extend Relationship illustrates that a use case can be extended by another use case under specific conditions. It is represented by a dashed arrow with the keyword “extend.” This relationship is useful for handling optional or exceptional behavior.

Example: Flight Booking System

- Use Cases: Book Flight, Select Seat
- Extend Relationship: The “Select Seat” use case may extend the “Book Flight” use case when the user wants to choose a specific seat, but it is an optional step.

4. Generalization Relationship: The Generalization Relationship establishes an “is-a” connection between two use cases, indicating that one use case is a specialized version of another. It is represented by an arrow pointing from the specialized use case to the general use case.

Example: Vehicle Rental System

- Use Cases: Rent Car, Rent Bike
- Generalization Relationship: Both “Rent Car” and “Rent Bike” are specialized versions of the general use case “Rent Vehicle.”

Generalization Relationship

Example 1: Use case diagram of an Online Shopping System:

Example 2: Use case diagram for the interaction of a client (the actor) within a restaurant (the system)

3.4 Design for Re-use

Design reuse is the inclusion of previously designed components (e.g., intellectual property) in software and hardware.

It is the design and development of a system from reusable software components. That is

1. Design for Reuse is the building of software from reusable components.
2. Design reuse is the process of building new software applications and tools by reusing previously developed designs. New features and functionalities may be added by incorporating minor changes.

Reusing designs makes it quicker, easier, and less expensive to design and build a new product.

Design reuse involves the use of designed modules, such as logic and data, to build a new and improved product. The reusable components, including code segments, structures, plans and reports, minimize implementation time and are less expensive. This avoids reinventing existing software by using techniques already developed and to create and test the software. Therefore, the ultimate goal of design reuse is to help the developers create better products maximizing its value with minimal resources, cost and effort.

Today, it is almost impossible to develop an entire product from scratch. Reuse of design becomes necessary to maintain continuity and connectivity. In the software field, the reuse of the modules and data helps save implementation time and increases the possibility of eliminating errors due to prior testing and use. Therefore, the effectiveness of design reuse is measured in terms of production, time, cost and quality of the product.

Software Reuse

Software Reuse includes the following:

- i. System reuse
 - ii. Application reuse
 - iii. Component Reuse
 - iv. Object or Function Reuse
1. System reuse: Complete systems, which may include several application programs.
 2. Application reuse: An application system product is a software system that can be adapted for different customers without changing the source code of the system. Application systems have generic features and so can be used/reused in different environments. Application are adapted by using built-in configuration mechanisms that allow the functionality of the system to be tailored to specific customer needs. An application may be reused either by incorporating it without change into other application families.

Benefits of application reuse:

1. More rapid deployment of a reliable system may be possible
2. It is possible to see what functionality is provided by the applications and so it is easier to

judge whether or not they are likely to be suitable.

3. Some development risks are avoided by using existing software. However, this approach has its own risks, as I discuss below.

4. Businesses can focus on their core activity without having to devote a lot of resources to IT systems development.

5. As operating platforms evolve, technology updates may be simplified as these are the responsibility of the COTS product

Problems of application system reuse:

1. Requirements usually have to be adapted to reflect the functionality

2. The COTS product may be based on assumptions that are practically impossible to change.

3. Choosing the right COTS system for an enterprise can be a difficult process, especially as many COTS products are not well documented.

4. There may be a lack of local expertise to support systems development.

5. The COTS product vendor controls system support and evolution.

3. Component Reuse: Components of an application from sub-systems to single objects may be reused. Components Reuse can be requirements specifications, design specifications, design documents, source code, user interfaces, user documentations or any other items associated with software.

i. code reuse: As the name implies, code reuse is about using existing code in different software products and not only for the ones that code was originally designed for.

Code Reusability is a practice of writing software code in a way that allows it to be reused in different parts of the same program or in different projects without having to be rewritten from scratch.

Code Reusability is fundamental to efficient software development, as it reduces redundancy, promotes maintainability, and speeds up the development process

There are two major types of code reuse:

1. Internal reuse – This is when code written internally by a developer team or business is reused for other projects.

2. External reuse – This is when some third-party tool or code is licensed and employed in a project. This can be tricky since costs will be involved, and time will be required to learn and implement the tool. Additionally, it creates a dependency upon an external tool which may lead to issues further down the line.

Techniques for Achieving Code Reusability

Below are different techniques of code reusability:

i. Modularization: Break down the code into smaller, independent modules (functions, classes, components) that can be reused in different contexts.

Example: A function that sorts an array of numbers can be reused in different parts of the application.

ii. Encapsulation and Abstraction: Encapsulate complex functionality within well-defined interfaces or classes. Use abstraction to hide implementation details.

Example: A payment gateway class that abstracts the process of making payments (like via credit card, PayPal) can be reused across different parts of the application.

iii. Inheritance (for Object-Oriented Programming): Reuse code by creating a new class that inherits functionality from an existing class.

Example: A Vehicle class with basic properties (speed, capacity) can be extended by other classes like Car, Truck, and Motorcycle to reuse its common features.

Generics and Templates: Use generics (in languages like Java, C#, C++) or templates (in C++) to create reusable code that works with any data type.

Example: A generic function to swap two values (could work with integers, strings, objects, etc.).

iv. Libraries and Frameworks: Reuse pre-built libraries, frameworks, and third-party components to handle common functionality (such as logging, authentication, or UI components).

Example: Using a framework like React for front-end development, which provides reusable components for building user interfaces.

Advantages of Code Reuse

Here are some of the advantages of Code Reusability:

- **Saves Time:** Code Reuse improves productivity and eliminates the need for rewriting pieces of code that are already perfectly functional and available to use. This also helps developers to focus their attention on writing original code to create new features which add value and improve the caliber of the software product. Businesses are often on a tight schedule that requires them to churn out applications at a fast pace in order to get an edge on their competitors; code for similar features can often be used across several projects to expedite their delivery.
- **Lower Cost:** Saving on time also allows the business to save on costs. Additionally, by utilizing pre-existing code, organizations can reduce expenses associated with app development, avoiding the need to hire more people and obtain additional resources.
- **Reduced Development Risks:** Often, the code being reused is tried and tested. Since it's already been to battle and survived, it's safe to say that the code will be highly reliable and free from defects. This guarantees a good user experience since the code is likely to run smoothly and be perfectly functional.
- **Prevents Code Bloat:** Bloated code is considered to be needlessly long, slow, and resource-intensive. To prevent the formation of undesired functionality or code bloat, it is crucial to reuse efficient and simple code, if it is available, across applications.

4. Object or Function Reuse: Software components that implement a single well-defined object or function may be reused. This makes programs more efficient, organized, and easier to maintain. Functions are self-contained pieces of code that can be called with a single command. It can be designed to accept arguments as input and return values. Parameters can

be used to make a function reusable and it can be passed different pieces of information, called arguments, each time the function is called.

Benefits of software reuse:

1. Accelerated development: Bringing a system to market as early as possible is often more important than overall development costs. Reusing software can speed up system production because both development and validation time may be reduced.
2. Effective use of specialists: Instead of doing the same work over and over again, application specialists can develop reusable software that encapsulates their knowledge.
3. Increased dependability: Reused software, which has been tried and tested in working systems, should be more dependable than new software. Its design and implementation faults should have been found and fixed.
4. Lower development costs: Development costs are proportional to the size of the software being developed. Reusing software means that fewer lines of code have to be written.
5. Reduced process risk: The cost of existing software is already known, whereas the costs of development are always a matter of judgment. This is an important factor for project management because it reduces the margin of error in project cost estimation. This is particularly true when relatively large software components such as subsystems are reused.
6. Standards compliance: Some standards, such as user interface standards, can be implemented as a set of reusable components. For example, if menus in a user interface are implemented using reusable components, all applications present the same menu formats to users. The use of standard user interfaces improves dependability because users make fewer mistakes when presented with a familiar interface.

Problems with software reuse:

1. Creating, maintaining, and using a component library: Populating a reusable component library and ensuring the software developers can use this library can be expensive. Development processes have to be adapted to ensure that the library is used.
2. Finding, understanding, and adapting reusable components: Software components have to be discovered in a library, understood and, sometimes, adapted to work in a new environment. Engineers must be reasonably confident of finding a component in the library before they include a component search as part of their normal development process.
3. Increased maintenance costs: If the source code of a reused software system or component is not available then maintenance costs may be higher because the reused elements of the system may become increasingly incompatible with system changes.
4. Lack of tool support: Some software tools do not support development with reuse. It may be difficult or impossible to integrate these tools with a component library system. The software process assumed by these tools may not take reuse into account. This is particularly true for tools that support embedded systems engineering, less so for object-oriented development tools.
5. Not-invented-here syndrome: Some software engineers prefer to rewrite components because they believe they can improve on them. This is partly to do with trust and partly

to do with the fact that writing original software is seen as more challenging than reusing other people's software.

Process of Development with Reuse

The development with reuse methodology involves establishing a well defined outline of the system requirements. The requirements are analysed and the components that can be reused are determined and selected. Based on the selected components the requirements are modified and the

architectural system is well established. The reusable components are determined and the components are specified based on the reusable components that are suitable for development of the system. The specified components are later incorporated in the development of the system.

w

Benefits of software reuse

Sn Benefits Explanation

1 Increased dependability Reused software, which has been tried and tested in working systems, should be more dependable than new software. Its design and implementation faults should have been found and fixed.

2. Reduced process risk The cost of existing software is already known, whereas the costs of development are always a matter of judgment. This is an important factor for project management because it reduces the margin of error in project cost estimation. This is particularly true when relatively large software components such as subsystems are reused

3. Effective use of specialists Instead of doing the same work over and over again, application specialists can develop reusable software that encapsulates their knowledge.

4 Standards compliance Some standards, such as user interface standards, can be implemented as a set of reusable components. For example, if menus in a user interface are implemented using reusable components, all applications present the same menu formats to users. The use of standard user interfaces improves dependability because users make fewer mistakes when presented with a familiar interface

5 Accelerated development Bringing a system to market as early as possible is often more important than overall development costs. Reusing software can speed up system production because both development and validation time may be reduced.

6 Increased maintenance If the source code of a reused software system or costs component is not available then maintenance costs may be higher because the reused elements of the system may become increasingly incompatible with system changes.

7 Lack of tool support Some software tools do not support development with reuse. It may be difficult or impossible to integrate these tools with a component library system. The software process assumed by these tools may not take reuse into account. This is particularly true for tools that support embedded systems engineering, less so for object-oriented development tools.

8 Not-invented-here Some software engineers prefer to rewrite components syndrome because they believe they can improve on them. This is partly to do with trust and partly to do with the fact that writing original software is seen as more challenging than reusing other people's software.

9 Creating, maintaining, and Populating a reusable component library and ensuring the using a component library software developers can use this library can be expensive. Development processes have to be adapted to ensure that the library is used.

10. Finding, understanding, and Software components have to be discovered in a library, adapting reusable understood and, sometimes, adapted to work in a new components environment. Engineers must be reasonably confident of finding a component in the library before they include a component search as part of their normal development process.

APPLICATION PROGRAMMING INTERFACE- API

An API is a set of programming code that enables data transmission between one software product and another.

APIs is a software intermediary, allowing two applications to communicate with each other using pre-established rules and protocols.

The API does not interact with the user but processes the data received from one program module and transmits the results back to the other module. The API can pass data back and forth between two applications, access the features or services of other applications, or create an application.

APIs are essential because they allow developers to easily integrate existing services or features from other applications without actually developing the service or feature. APIs increase development speed by not having to develop code from scratch

How do APIs work

The working principle of an API is commonly expressed through the request-response communication between a client and a server. The client is any front-end application that a user interacts with. The server is in charge of backend logic and database operations. An API works as a middle layer between the client and the server, making it possible to send data requests and responses.

Programming languages used for APIs are Java, JavaScript, Perl, Python, and Ruby.

Types of API

There are different types of APIs that can be categorized based on the ways they are available for use and according to their initial design purposes.

APIs by Release policies

In terms of release policies, APIs can be private, partner, and public.

1. Private APIs: These are designed to improve organizational solutions and services. In-house developers or contractors may use these APIs to integrate or applications as well as build new systems. Even if apps are publicly accessible, the interface itself remains available only for those working directly with the API publisher.
2. Partner APIs: This type of API is openly promoted but shared with business partners who have signed an agreement with the publisher.
3. Public APIs: Also known as external APIs. These APIs are available for any third-party developers. There are two types of public APIs – open and commercial APIs
 - i. Open public APIs are those with all features (documentations) made public or available for use without restrictive terms and conditions.
 - ii. Commercial API users pay subscription fees or utilize APIs on a pay-as-you-go basis. A popular approach among publishers is to offer free trials so clients can evaluate APIs before purchasing subscriptions.

APIs by use cases

APIs can be classified according to the systems for which they are designed. These are:

1. Database APIs. Database APIs enable communication between an application and a database management system.
2. Operating systems APIs. This group of APIs defines how applications use the resources and services of operating systems. Those building apps for the iOS mobile operating system use Cocoa Touch – a modified version of Cocoa.
3. Remote APIs. Remote APIs define standards of interaction for applications running on different machines. In other words, one software product accesses resources located outside the device that requests them. Since two remotely located applications are connected over a communications network, particularly the internet, most remote APIs are written based on web standards. Java Examples are Database Connectivity API and Java Remote Method Invocation API
4. Web APIs. This API class is the most common. Web APIs provide machine-readable data and functionality transfer between web-based systems which represent client-server architecture. These APIs mainly deliver requests from web applications and responses from servers using Hypertext Transfer Protocol (HTTP).

Key components of an API

To successfully execute an API, the following components are needed:

1. API client: A user can be an API client that initiates an API request, or a request is automatically activated by an external event or notification from a service or application.

The API client can be triggered by a user clicking on a button, application, or service. The API client makes it easier for a person to use while hiding the complexities of the backend details.

2. API requests: An API request is a message sent to an application asking a server for information or a service. The Representational State Transfer (REST) APIs are commonly used. The sub-components or parameters that make up an API requests are:

i. Endpoint: An endpoint is a dedicated Uniform Resource Locator (URL) that points to the location of a resource on a server.

ii. Request method: The request methods are the specific operations the client wants to perform on the URL resource. REST API uses the HTTP method that can perform the following actions:

- GET – retrieves data from a server; see Figure 2.
- POST – adds new data saved to a URL resource on a server; See Figure 3.
- PUT – replaces an entire resource with new information
- PATCH – is used to partially update an existing URL resource with additional information
- DELETE – used to remove data from a database

iii. Parameters: These are the variables passed to an API endpoint to provide explicit instructions for the API server to process. The parameters can be included as part of the API request in the URL query string or in the request body field, as shown in Figure 3. In Figure 4, notice how the parameters are included in the HTTP endpoint URL sent to an API server on a web server.

iv. Request headers: The request headers provide essential information for a server to process the request, and the header information is in the message body. Headers give the following information:

- Specifies the format the data will be sent in, such as the JavaScript Object Notation (JSON) format
- Identifies the API version to call
- Provides an API key for authentication
- Dictates the behavior of the server in handling the request
- Provides metadata information about the request or response
- Contains information about the request method used
- Includes information on the content type of the requested payload

3. API server: The API server is software that resides directly on a server. The API server sits between the client and the data source, so web APIs sit between a user application as the web server. Once an API client creates an API request, the request goes to the appropriate endpoint on the API server for processing. The API server handles authentication, validates the inputted data, retrieves or manipulates the data from a database, and returns the appropriate response to the client. See Figure 5

4. API response: The API server generates the API response that returns to the API client.

The API response can respond in multiple ways depending on what was in the API request. An API response provides the following information:

- i. Status code :The status code informs the client of the results of the submitted API request. The codes help the client understand what happened with the request. Code 200 signifies the server successfully returned the requested data, and code 201 indicates the server successfully created a new resource. The code 404, which we have probably all experienced, means Not Found, so no action was taken by the server.
- ii. Response headers: Response headers provide additional information about the server's response.
- iii. Body: The response body is the data that is returned by the API server based on the client's request.

Step-by-step guide on how to use an API

To implement a successful API application, you need to follow a similar step-by-step process:

1. Develop an API strategy to deliver business profit or value : This is the goal of the purported API application a business wants to develop. Will the API increase revenues, enhance operational efficiency, or use existing data or technology to generate additional revenue?
2. Designate a data source for the API and create an API diagram: Create a data model and the activity required to interact with the data sources. To develop the API, the developers must know the requirements, parameters must be included in the endpoint with the HTTP methods, and the data the API needs to retrieve the database results
3. Assess your business network: Assessing your business network to select an API solution that can easily integrate within your network and software resources
4. Define API requirements: The expectation of what the API should do must be clearly defined. The API requirement must be tied back to the original API strategy.
5. Select an API data exchange architecture: There are multiple types of APIs such as , REST and SOAP etc, The REST API is easy to use, faster than typical web services, and can return results using different data formats..
6. Choose an API authentication method: REST APIs' typical authentication method are API keys, which can be sent in a query string or request header..
7. Creating an API specification and developing API documentation: An abundance of API tools are available to help keep your API application updated and documented. You want your API documentation to be easily interpreted and understood by developers.
8. Keeping the latest API updated with API versioning: You want your users and developers to know when a new API is released. As your API app is updated, the documentation needs to reflect a new version has been released.
9. API deployment and development: Using a Continuous Integration/Continuous Delivery process to automate application deployment gets API apps deployed faster without human intervention.

10. Monitoring an API app: You have created an API application that is properly functioning and meeting the expected metrics, indicating the API has met the initial goal outlined in the API strategy. To ensure your API application is continuously working, you can invest in API monitoring software to detect power or network outages.

The goal of monitoring API software is to minimize downtime by addressing issues before they escalate, identifying issues that may impact the API's performance, and resolving any problematic issues that could affect the customer negatively or cause potential revenue loss.

The full implementation of a successful API involves multiple steps that become the foundation for your API app. Even though the development and deployment processes are undoubtedly critical in building and running an API, you want to ensure that as long as the API is online, it's performing optimally. Therefore, implementing an API entails constant monitoring and applying practical updates as business processes evolve. The automated API tools that help with API creation and monitoring are essential in ensuring the API developed in the API implementation process constantly meets the intended goals outlined in the API strategy.

Examples of API

1. Google Maps. It is no secret that Google is among the tech giants, and they have set the standards in the way other companies operate. Most websites that have an integrated map are using the Google Maps APIs.
2. Vulkan. Vulkan is a cross-platform API that works on the operating system level. It enables developers to create high-quality, real-time graphics in applications and drives the communication between an app and a graphical processing unit (GPU).
3. Skyscanner Flight Search. It lets travelers search for flights at the best rates from its database of prices.
4. WeatherAPI. This is a free geolocation and weather information provider with lots of different APIs that provide access to the weather forecast, IP lookup, sports, astronomy, geolocation, and other data..
5. Amadeus Travel APIs. Amadeus, another large GDS,
6. Yelp API. This is a GraphQL API that provides users with recommendations and reviews of the best restaurants. It uses JSON as the data exchange format.

TOPIC 3:

SOFTWARE ENGINEERING METHODOLOGIES

Software development methodologies are frameworks used to structure, plan, and manage the software development process. Its goal is to ensure the delivery of software in accordance with project specifications, within time and budget, and with minimized project risks.

Each methodology offers a unique approach to managing the software development life cycle, defining project steps, roles, responsibilities, activities, communication standards, and deliverables. The diversity in methodologies allows companies to choose the one that better aligns with their specific needs and project requirements.

Current trends in software development methodologies

There are two basic Software Development Methodologies.

1. Agile Methodologies
2. Sequential Methodologies

Since its inception in 2001, Agile has been gaining traction because of its responsiveness to constant change and continuous delivery of value to customers. According to Agile Report , 71% of companies use Agile in their software development lifecycle.

However, some industries still use sequential methodologies that provide a predictable and structured approach with thorough preliminary planning

Sequential methodologies

Sequential or plan-driven methodologies break up the software development cycle into consecutive, standalone phases. These models prioritize gathering detailed project requirements, planning the entire development process, and establishing deliverables for each phase before starting the development.

Advantages of sequential methodologies

- i. It is a straightforward, easy-to-understand workflow simplifies project management
- ii. It facilitates project stages tracking for project sponsors
- iii. It allow developers to better focus on the project's goals
- iv. It has clear project scope and predictable timeline, cost, and deliverables as a result of thorough planning and early definition of requirements
- v. Detailed documentation makes tracking to the project's progress easier
- vi. It simplifies software maintenance and further modernization

Limitations of sequential methodologies

- i. Extensive requirements gathering leads to a prolonged discovery phase
- ii. Stakeholders are mainly involved at the beginning of the project, which can result in lower end-user satisfaction
- iii. The inflexible step-by-step process contributes to lengthy project duration
- iv. Addressing evolving requirements is challenging since it entails going back to the initial stage of development and re-elaborating requirements

Types of sequential methodologies

1. Waterfall: The waterfall development methodology follows a linear workflow, progressing from top to bottom through several stages that focus on distinct goals. Once the development process starts, no extensions or alterations to the project scope are allowed, so all requirements must be elicited during the planning stage. Developers must complete each stage before moving to the next one, meaning the software is implemented as the finished product at the end of the project.

Suitable for:

- i. Projects with well-defined, unchanging requirements and objectives
- ii. Projects where timeline and budget control is critical

Benefits of waterfall Methodology

- With the reason of simplicity and sequence system it is very easy to use it.
- User can understand it quickly because simplicity is the main quality of this method.
- It defines the complete information about every step.
- This model is very helpful in planning and scheduling of projects.
- It reduces the cost of Error correctness.
- Importantly it increases the possibility that the system fulfill the customer needs

Limitations of waterfall Methodology

- Expectations become very quickly in this process which is not correct.
- Risk factors not involved in this model which is very necessary part for every model.
- In this method training sessions of user is not necessary.
- We cannot find any error till time testing of software not done.
- Between user and developer the interaction gap is low.

Major outputs in a development project which follows the waterfall model are following:

Sr.No. Activity Phase Output Documents

- 1 Requirement analysis SRS documents
- 2 Design phase Data flow diagram
- 3 Coding Program code
- 4 Unit Testing Module test report
- 5 System testing System test report
- 6 Acceptance testing Final system documentation

2. V-model The V-model is an extension of the Waterfall methodology that emphasizes testing in parallel with the development process. Each subsequent development phase can only start after testing for the previous stage, thus reducing the probability of defects and errors in the final product and ensuring high software quality. However, the focus on the testing activities can make the development time-consuming and costly.

Suitable for:

- Projects that can use Waterfall but demand higher levels of quality and reliability

Software development life cycle in the V-model

3. Rapid application development

RAD promotes short development cycles and rapid delivery of functional software. This model focuses on iterative app development, where the project is broken down into smaller modules developed in parallel by different teams. The RAD model implies gathering requirements through focus groups, prototyping and early user testing to get customer feedback and accommodate possible requirement changes before the development starts. It also emphasizes

reusing software components across the system, adhering to strict upfront dependencies planning and precise project scheduling and coordinating, extensive requirements elaboration and software design efforts, as well as a team of highly experienced analysts and architects.

Suitable for:

- i. Projects where requirements can be identified upfront
- ii. Systems that can be modularized

Process model for Rapid application development

4. Spiral The spiral model combines elements of sequential and iterative approaches, which makes modifying requirements easier than in the classic Waterfall model. The development team works on the four software development stages in cycles, or spirals, and improves the product each time. Each cycle can take up to several months and starts with identifying project goals and planning, followed by risk analysis and management, which reduces the likelihood of errors and issues. Then, programmers build software based on the requirements gathered during the previous iteration. The software is evaluated at the end of each cycle to ascertain its alignment with the defined requirements. The project stakeholders then decide whether to proceed to the next iteration or complete the project, thus predicting the number of iterations required to finish the project with the spiral model can be challenging, which leads to uncertainty in the project timeline and cost.

Suitable for:

- i. Complex and large projects characterized by uncertainty or changing requirements
- ii. Projects where risk management is critical

Software development life cycle in the Spiral model

5. Incremental The incremental model breaks the development process into smaller, stand-alone parts, called increments. Each iteration can last up to several months, during which the team builds a particular feature set – a portion of the complete system’s functionality. End-users get access to a fully functional software subset with the implementation of the first increment while the development of further increments continues until the entire product is completed. With incremental development, potential issues can be detected early in the development process, minimizing the probability of costly errors at later stages. This model requires careful and strict increment planning to make it consistent and valuable to the end-users.

Suitable for:

- i. Projects with a lengthy development schedule that focus on bringing value as early as possible
- ii. The product needs to enter the market quickly
- iii. When the system allows for easy modularity

A, B, and C are modules of software products that are incrementally developed and delivered

Agile Methodologies

The main focus of agile methodologies remains on creating short workable increments delivering value to their customers. The process leaders continuously Evaluate the requirements, plans, and

results throughout the life cycle. Agile consists of multiple methodologies.

Extreme Programming

Extreme Programming is an agile software development framework. It is aimed at producing high-quality software that improves the quality of life for the development team. As the name of the framework suggests, Extreme Programming is very specific regarding the appropriate practices for software development.

Kanban

Kanban is a Japanese word that roughly translates to a signboard or billboard. Originated from the body shops of Toyota manufacturing plants in Japan, it was quickly adopted by the software developers and project managers due to its effectiveness.

Lean

As the name suggests, Lean methodology aims at removing the clutter around the product life cycle and making the process as lean as possible. This helps put priority on the tasks that deserve it and also speeds up the process of the entire lifecycle.

Scrum

Scrum is also a framework development model that is inspired by Agile principles. Scrum works best for teams of 10 or fewer members. Each member of the team is assigned a task to be completed

in a timed box iteration called sprints (shorter than a month). The progress of tasks is assessed by daily meetings of 15 minutes or less known as daily scrums. The completion of the sprint is followed by meeting with the stakeholders to demonstrate the tasks completed.

Crystal

Crystal becomes a little subjective compared to the other framework models we have studied till now. Rather than focus on tools and technologies, Crystal aims at focusing on people and communication between them. Crystal is built on two core beliefs:

- Teams can find ways on their own to improve and optimize their workflow
- Since the requirement of each project is unique and different, the decision-making power should be left with the project teams.

12 principles of the Agile approach

Advantages of Agile development methodologies

- i. Short iterations enable the team to release software frequently, quickly delivering value to business users
- ii. If the project requirements change, the team can quickly adapt and deliver software in line with the new needs within a few iterations
- iii. Effective communication between the development team and business stakeholders ensures better alignment of the product with the business users' expectations

Limitations of Agile development methodologies

- i. With frequently changing requirements and many small sprints, assessing the project's overall deadline can be challenging

- ii. Requires high level of discipline and engagement to avoid extensive communication overheads
- iii. Limited documentation can pose challenges to knowledge sharing or regulatory compliance

Paper 6 of 9 — CSC 411 handout

Level 400 • First Semester • Handout

Software Engineering Lecture Notes. Dr. David A. Oyemade

SOFTWARE ENGINEERING

LECTURE NOTES

Dr. David A. Oyemade

1

Software Engineering Lecture Notes. Dr. David A. Oyemade

CSC 411: SOFTWARE ENGINEERING: (3 Units)

Objectives/Learning Outcomes

At the end of this course you will understand the following aspects of Software Engineering and you will be able to apply them to practical software projects:

Software Design: Software architecture

Design Patterns

Object Oriented Analysis & Design

Design for re-use.

Using APIs: API programming Class browsers and related tools

Component based computing.

Software tools and Environment:

Requirements analysis and design

Software Modeling Tools,

Testing tools, Tool integration mechanisms.

2

Software Engineering Lecture Notes. Dr. David A. Oyemade

1. Introduction

Getting started with software engineering

Objectives

To introduce software engineering and to explain its importance

To set out the answers to key questions about software engineering

To introduce ethical and professional issues and to explain why they are of concern to software engineers

Topics covered

FAQs about software engineering

Software engineering

The economies of ALL developed nations are dependent on software

More and more systems are software controlled

Software engineering is concerned with theories, methods and tools for professional software development

Software engineering expenditure represents a significant fraction of GNP in all developed countries

3

Software Engineering Lecture Notes. Dr. David A. Oyemade

FAQs about software engineering

What is software?

What is software engineering?

What is the difference between software engineering and computer science?

What is the difference between software engineering and system engineering?

What is a software process?

What is a software process model?

FAQs about software engineering

What are the costs of software engineering?

What are software engineering methods?

What is CASE (Computer-Aided Software Engineering)

What are the attributes of good software?

What are the key challenges facing software engineering?

What is software?

Computer programs and associated documentation

Software products may be developed for a particular customer or may be developed for a general market

Software products may be

- Generic - developed to be sold to a range of different customers
- Custom - developed for a single customer according to their specification

What is software engineering?

Software engineering is an engineering discipline which is concerned with all aspects of software production

Software engineers should adopt a systematic and organised approach to their work and use appropriate tools and techniques depending on the problem to be solved, the development constraints and the resources available

What is the difference between software engineering and computer science?

Computer science is concerned with theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software

4

Software Engineering Lecture Notes. Dr. David A. Oyemade

Computer science theories are currently insufficient to act as a complete underpinning for software engineering

What is a software process?

A set of activities whose goal is the development or evolution of software

Generic activities in all software processes are:

- Specification - what the system should do and its development constraints
- Development - production of the software system
- Validation - checking that the software is what the customer wants
- Evolution - changing the software in response to changing demands

What is a software process model?

A simplified representation of a software process, presented from a specific perspective

Examples of process perspectives are

- Workflow perspective - sequence of activities
- Data-flow perspective - information flow
- Role/action perspective - who does what

Generic process models

- Waterfall
- Evolutionary development
- Formal transformation
- Integration from reusable components

What are software engineering methods?

Structured approaches to software development which include system models, notations, rules, design advice and process guidance

Model descriptions

- Descriptions of graphical models which should be produced

Rules

- Constraints applied to system models

Recommendations

- Advice on good design practice

Process guidance

- What activities to follow

What is CASE (Computer-Aided Software Engineering)

Software systems which are intended to provide automated support for software process activities. CASE systems are often used for method support

Upper-CASE

- Tools to support the early process activities of requirements and design

Lower-CASE

5

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Tools to support later activities such as programming, debugging and testing

What are the attributes of good software?

The software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable

Maintainability

- Software must evolve to meet changing needs

Dependability

- Software must be trustworthy

Efficiency

- Software should not make wasteful use of system resources

Usability

- Software must be usable by the users for which it was designed

What are the key challenges facing software engineering?

Coping with legacy systems, coping with increasing diversity and coping with demands for reduced delivery times

Legacy systems

- Old, valuable systems must be maintained and updated

Heterogeneity

- Systems are distributed and include a mix of hardware and software

Delivery

- There is increasing pressure for faster delivery of software

Key points

Software engineering is an engineering discipline which is concerned with all aspects of software production.

Software products consist of developed programs and associated documentation. Essential product attributes are maintainability, dependability, efficiency and usability.

The software process consists of activities which are involved in developing software products.

Basic activities are software specification, development, validation and evolution.

Methods are organised ways of producing software. They include suggestions for the process to be followed, the notations to be used, rules governing the system descriptions which are produced and design guidelines.

CASE tools are software systems which are designed to support routine activities in the software process such as editing design diagrams, checking diagram consistency and keeping track of program tests which have been run.

6

Software Engineering Lecture Notes. Dr. David A. Oyemade

2. Software Processes

Informal Scenarios

Help understand the problem

Come up with questions on requirements

Help constrain architecture

Software Processes

Coherent sets of activities for specifying, designing, implementing and testing software systems

Objectives

To introduce software process models

To describe a number of different process models and when they may be used

To describe process models for requirements engineering, software development, testing and evolution

Topics covered

Software process models

Process iteration

Software specification

Software design and implementation

Software validation

Software evolution

Automated process support

The software process

A structured set of activities required to develop a software system

- Specification
- Design
- Validation
- Evolution

7

Software Engineering Lecture Notes. Dr. David A. Oyemade

A software process model is an abstract representation of a process. It presents a description of a process from some particular perspective

Generic software process models

The waterfall model

- Separate and distinct phases of specification and development

Evolutionary development

- Specification and development are interleaved

Formal systems development

- A mathematical system model is formally transformed to an implementation

Reuse-based development

- The system is assembled from existing components

Waterfall model

8

Software Engineering Lecture Notes. Dr. David A. Oyemade

Waterfall model phases

Requirements analysis and definition

System and software design

Implementation and unit testing

Integration and system testing

Operation and maintenance

The drawback of the waterfall model is the difficulty of accommodating change after the process is underway

Waterfall model problems

Inflexible partitioning of the project into distinct stages

This makes it difficult to respond to changing customer requirements

Therefore, this model is only appropriate when the requirements are well-understood

Evolutionary development

Exploratory development

- Objective is to work with customers and to evolve a final system from an initial outline specification. Should start with well-understood requirements

Throw-away prototyping

- Objective is to understand the system requirements. Should start with poorly understood requirements

Evolutionary development

9

Software Engineering Lecture Notes. Dr. David A. Oyemade

Evolutionary development

Problems

- Lack of process visibility
- Systems are often poorly structured
- Special skills (e.g. in languages for rapid prototyping) may be required

Applicability

- For small or medium-size interactive systems
- For parts of large systems (e.g. the user interface)
- For short-lifetime systems

Formal systems development

Based on the transformation of a mathematical specification through different representations to an executable program

Transformations are 'correctness-preserving' so it is straightforward to show that the program conforms to its specification

Embodied in the 'Cleanroom' approach to software development

Formal systems development

Reuse-oriented development

Based on systematic reuse where systems are integrated from existing components or COTS (Commercial-off-the-shelf) systems

Process stages

- Component analysis
- Requirements modification
- System design with reuse
- Development and integration

This approach is becoming more important but still limited experience with it

10

Software Engineering Lecture Notes. Dr. David A. Oyemade

Reuse-oriented development

Process iteration

System requirements ALWAYS evolve in the course of a project so process iteration where earlier stages are reworked is always part of the process for large systems

Iteration can be applied to any of the generic process models

Two (related) approaches

- Incremental development
- Spiral development

Incremental development

Rather than deliver the system as a single delivery, the development and delivery is broken down into increments with each increment delivering part of the required functionality

User requirements are prioritised and the highest priority requirements are included in early increments

Once the development of an increment is started, the requirements are frozen though requirements for later increments can continue to evolve

Incremental development

11

Software Engineering Lecture Notes. Dr. David A. Oyemade

Incremental development advantages

Customer value can be delivered with each increment so system functionality is available earlier

Early increments act as a prototype to help elicit requirements for later increments

Lower risk of overall project failure

The highest priority system services tend to receive the most testing

Extreme programming

New approach to development based on the development and delivery of very small increments of functionality

Relies on constant code improvement, user involvement in the development team and pairwise programming

Software specification

The process of establishing what services are required and the constraints on the system's operation and development

Requirements engineering process

- Feasibility study
- Requirements elicitation and analysis
- Requirements specification
- Requirements validation

The requirements engineering process

12

Software Engineering Lecture Notes. Dr. David A. Oyemade

Software design and implementation

The process of converting the system specification into an executable system

Software design

- Design a software structure that realises the specification

Implementation

- Translate this structure into an executable program

The activities of design and implementation are closely related and may be inter-leaved

Design process activities

Architectural design

Abstract specification

Interface design

Component design

Data structure design

Algorithm design

The software design process

13

Software Engineering Lecture Notes. Dr. David A. Oyemade

Design methods

Systematic approaches to developing a software design

The design is usually documented as a set of graphical models

Possible models

- Data-flow model
- Entity-relation-attribute model
- Structural model
- Object models

Programming and debugging

Translating a design into a program and removing errors from that program

Programming is a personal activity - there is no generic programming process

Programmers carry out some program testing to discover faults in the program and remove these faults in the debugging process

The debugging process

Software validation

Verification and validation is intended to show that a system conforms to its specification and meets the requirements of the system customer

Involves checking and review processes and system testing

System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system

14

Software Engineering Lecture Notes. Dr. David A. Oyemade

The testing process

Testing stages

Unit testing

- Individual components are tested

Module testing

- Related collections of dependent components are tested

Sub-system testing

- Modules are integrated into sub-systems and tested. The focus here should be on interface testing

System testing

- Testing of the system as a whole. Testing of emergent properties

Acceptance testing

- Testing with customer data to check that it is acceptable

Testing phases

15

Software Engineering Lecture Notes. Dr. David A. Oyemade

Software evolution

Software is inherently flexible and can change.

As requirements change through changing business circumstances, the software that supports the business must also evolve and change

Although there has been a demarcation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new

System evolution

Automated process support (CASE)

Computer-aided software engineering (CASE) is software to support software development and evolution processes

Activity automation

- Graphical editors for system model development
- Data dictionary to manage design entities
- Graphical UI builder for user interface construction
- Debuggers to support program fault finding
- Automated translators to generate new versions of a program

16

Software Engineering Lecture Notes. Dr. David A. Oyemade

Key points

Software processes are the activities involved in producing and evolving a software system. They are represented in a software process model

General activities are specification, design and implementation, validation and evolution

Generic process models describe the organisation of software processes

Iterative process models describe the software process as a cycle of activities

Requirements engineering is the process of developing a software specification

Design and implementation processes transform the specification to an executable program

Validation involves checking that the system meets to its specification and user needs

Evolution is concerned with modifying the system after it is in use

CASE technology supports software process activities

17

Software Engineering Lecture Notes. Dr. David A. Oyemade

2 Design with Reuse

- Building software from reusable components.
- In most engineering disciplines, systems are designed by composing existing components that have been used in other systems
- Software engineering has been more focused on original development but it is now recognised that to achieve better software, more quickly and at lower cost, we need to adopt a design process that is based on systematic reuse

Reuse-based software engineering

- Application system reuse
 - The whole of an application system may be reused either by incorporating it without change into other systems (COTS reuse) or by developing application families
- Component reuse
 - Components of an application from sub-systems to single objects may be reused
- Function reuse
 - Software components that implement a single well-defined function may be reused

Reuse practice

- Application system reuse
 - Widely practised as software systems are implemented as application families. COTS reuse is becoming increasingly common

1

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Component reuse
 - Now seen as the key to effective and widespread reuse through component-based software engineering. However, it is still relatively immature
- Function reuse
 - Common in some application domains (e.g. engineering) where domain-specific libraries of reusable functions have been established

Benefits of reuse

- Increased reliability
 - Components exercised in working systems
- Reduced process risk
 - Less uncertainty in development costs
- Effective use of specialists
 - Reuse components instead of people

- Standards compliance
- Embed standards in reusable components
- Accelerated development
- Avoid original development and hence speed-up production

Requirements for design with reuse

2

Software Engineering Lecture Notes. Dr. David A. Oyemade

- It must be possible to find appropriate reusable components
- The re-user of the component must be confident that the components will be reliable and will behave as specified
- The components must be documented so that they can be understood and, where appropriate, modified

Reuse problems

- Increased maintenance costs
- Lack of tool support
- Not-invented-here syndrome
- Maintaining a component library
- Finding and adapting reusable components

Generator-based reuse

- Program generators involve the reuse of standard patterns and algorithms
- These are embedded in the generator and parameterised by user commands. A program is then automatically generated
- Generator-based reuse is possible when domain abstractions and their mapping to executable code can be identified
- A domain specific language is used to compose and control these abstractions

Types of program generator

- Application generators for business data processing
- Parser and lexical analyser generators for language processing
- Code generators in CASE tools

3

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Generator-based reuse is very cost-effective but its applicability is limited to a relatively small number of application domains
- It is easier for end-users to develop programs using generators compared to other component-based approaches to reuse

Reuse through program generation

Component-based development

- Component-based software engineering (CBSE) is an approach to software development that relies on reuse
- It emerged from the failure of object-oriented development to support effective reuse. Single object classes are too detailed and specific
- Components are more abstract than object classes and can be considered to be stand-alone service providers

Components

- Components provide a service without regard to where the component is executing or its programming language

4

Software Engineering Lecture Notes. Dr. David A. Oyemade

- A component is an independent executable entity that can be made up of one or more executable objects
- The component interface is published and all interactions are through the published interface
- Components can range in size from simple functions to entire application systems

Component interfaces

- Provides interface
- Defines the services that are provided by the component to other components
- Requires interface
- Defines the services that specifies what services must be made available for the component to execute as specified

Printing services component

5

Software Engineering Lecture Notes. Dr. David A. Oyemade

Component abstractions

- Functional abstraction
- The component implements a single function such as a mathematical function
- Casual groupings
- The component is a collection of loosely related entities that might be data declarations, functions, etc.
- Data abstractions
- The component represents a data abstraction or class in an object-oriented

language

- Cluster abstractions
- The component is a group of related classes that work together
- System abstraction
- The component is an entire self-contained system

CBSE processes

- Component-based development can be integrated into a standard software process by incorporating a reuse activity in the process

6

Software Engineering Lecture Notes. Dr. David A. Oyemade

- However, in reuse-driven development, the system requirements are modified to reflect the components that are available
- CBSE usually involves a prototyping or an incremental development process with components being 'glued together' using a scripting language

An opportunistic reuse process

Development with reuse

CBSE problems

- Component incompatibilities may mean that cost and schedule savings are less than expected
- Finding and understanding components
- Managing evolution as requirements change in situations where it may be impossible to change the system components

7

Software Engineering Lecture Notes. Dr. David A. Oyemade

Application frameworks

- Frameworks are a sub-system design made up of a collection of abstract and concrete classes and the interfaces between them
- The sub-system is implemented by adding components to fill in parts of the design and by instantiating the abstract classes in the framework
- Frameworks are moderately large entities that can be reused
- Framework classes
- System infrastructure frameworks
- Support the development of system infrastructures such as communications, user interfaces and compilers
- Middleware integration frameworks
- Standards and classes that support component communication and

information exchange

- Enterprise application frameworks
- Support the development of specific types of application such as telecommunications or financial systems

Extending frameworks

- Frameworks are generic and are extended to create a more specific application or sub-system
- Extending the framework involves
 - Adding concrete classes that inherit operations from abstract classes in the framework
 - Adding methods that are called in response to events that are recognised by the framework
- Problem with frameworks is their complexity and the time it takes to use them effectively
- Model-view controller

8

Software Engineering Lecture Notes. Dr. David A. Oyemade

- System infrastructure framework for GUI design
- Allows for multiple presentations of an object and separate interactions with these presentations

MVC framework involves the instantiation of a number of Model-view controller

COTS product reuse

- COTS - Commercial Off-The-Shelf systems
- COTS systems are usually complete application systems that offer an API (Application Programming Interface)
- Building large systems by integrating COTS systems is now a viable development strategy for some types of system such as E-commerce systems
- COTS system integration problems
 - Lack of control over functionality and performance
 - COTS systems may be less effective than they appear
 - Problems with COTS system inter-operability
 - Different COTS systems may make different assumptions that means integration is difficult
 - No control over system evolution
 - COTS vendors not system users control evolution
 - Support from COTS vendors

- COTS vendors may not offer support over the lifetime of the product

Component development for reuse

- Components for reuse may be specially constructed by generalising existing components

9

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Component reusability
- Should reflect stable domain abstractions
- Should hide state representation
- Should be as independent as possible
- Should publish exceptions through the component interface

There is a trade-off between reusability and usability.

- The more general the interface, the greater the reusability but it is then more complex and hence less usable

Reusable components

- The development cost of reusable components is higher than the cost of specific equivalents. This extra reusability enhancement cost should be an organization rather than a project cost
- Generic components may be less space-efficient and may have longer execution times than their specific equivalents

Reusability enhancement

- Name generalisation
- Names in a component may be modified so that they are not a direct reflection of a specific application entity
- Operation generalisation
- Operations may be added to provide extra functionality and application specific operations may be removed
- Exception generalisation
- Application specific exceptions are removed and exception management added to increase the robustness of the component

10

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Component certification
- Component is certified as reusable

Reusability enhancement process

Design patterns

- A design pattern is a way of reusing abstract knowledge about a problem and its solution
- A pattern is a description of the problem and the essence of its solution
- It should be sufficiently abstract to be reused in different settings
- Patterns often rely on object characteristics such as inheritance and polymorphism
- Pattern elements
 - Name
 - A meaningful pattern identifier
 - Problem description
 - Solution description
 - Not a concrete design but a template for a design solution that can be instantiated in different ways
 - Consequences
 - The results and trade-offs of applying the pattern

11

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Multiple displays
- Key points
- Design with reuse involves designing software around good design and existing components
- Advantages are lower costs, faster software development and lower risks
- Component-based software engineering relies on black-box components with defined requires and provides interfaces
- COTS product reuse is concerned with the reuse of large, off-the-shelf systems

Key points

- Software components for reuse should be independent, should reflect stable domain abstractions and should provide access to state through interface operations
- Application families are related applications developed around a common core
- Design patterns are high-level abstractions that document successful design solutions

12

Software Engineering Lecture Notes. Dr. David A. Oyemade

Assignment: Model your final year project using object oriented analysis and design (OOAD). Use CASE tools for the modeling and an object oriented programming

language such as Java, C# or C++ for the implementation.

(30/6/2016).

13

Software Engineering Lecture Notes. Dr. David A. Oyemade

3 OBJECT ORIENTED DESIGN

Characteristics of OOD

Objects are abstractions of real-world or system entities and manage themselves

- Objects are independent and encapsulate state and representation information.
- System functionality is expressed in terms of object services
- Shared data areas are eliminated. Objects communicate by message passing
- Objects may be distributed and may execute sequentially or in parallel

Advantages of OOD

- Easier maintenance. Objects may be understood as stand-alone entities
- Objects are appropriate reusable components
- For some systems, there may be an obvious mapping from real world entities to system objects

Object-oriented development

- Object-oriented analysis, design and programming are related but distinct
- OOA is concerned with developing an object model of the application domain
- OOD is concerned with developing an object-oriented system model to implement requirements
- OOP is concerned with realising an OOD using an OO programming language such as Java or C++

Objects and object classes

- Objects are entities in a software system which represent instances of real-world and system entities
- Object classes are templates for objects. They may be used to create objects
- Object classes may inherit attributes and services from other object classes

1

Software Engineering Lecture Notes. Dr. David A. Oyemade

Objects

The Unified Modeling Language

- Several different notations for describing object-oriented designs were proposed

in the 1980s and 1990s

- The Unified Modeling Language is an integration of these notations
- It describes notations for a number of different models that may be produced during OO analysis and design
- It is now a de facto standard for OO modelling

Employee object class (UML)

Object communication

- Conceptually, objects communicate by message passing.

Messages

- The name of the service requested by the calling object.
- Copies of the information required to execute the service and the name of a holder for the result of the service.

2

Software Engineering Lecture Notes. Dr. David A. Oyemade

o In practice, messages are often implemented by procedure calls

- Name = procedure name.
- Information = parameter list.

Message examples

```
// Call a method associated with a buffer
// object that returns the next value
// in the buffer
v = circularBuffer.Get () ;
// Call the method associated with a
// thermostat object that sets the
// temperature to be maintained
thermostat.setTemp (20) ;
```

Generalisation and inheritance

- Objects are members of classes which define attribute types and operations
- Classes may be arranged in a class hierarchy where one class (a super-class) is a generalisation of one or more other classes (sub-classes)
- A sub-class inherits the attributes and operations from its super class and may add new methods or attributes of its own
- Generalisation in the UML is implemented as inheritance in OO programming

languages

3

Software Engineering Lecture Notes. Dr. David A. Oyemade

A generalisation hierarchy

Advantages of inheritance

- It is an abstraction mechanism which may be used to classify entities
- It is a reuse mechanism at both the design and the programming level
- The inheritance graph is a source of organisational knowledge about domains and systems

UML associations

- Objects and object classes participate in relationships with other objects and object classes
- In the UML, a generalised relationship is indicated by an association
- Associations may be annotated with information that describes the association
- Associations are general but may indicate that an attribute of an object is an associated object or that a method relies on an associated object

4

Software Engineering Lecture Notes. Dr. David A. Oyemade

An association model

Concurrent objects

- The nature of objects as self-contained entities make them suitable for concurrent implementation
- The message-passing model of object communication can be implemented directly if objects are running on separate processors in a distributed system

Servers and active objects

There are two kinds of concurrent objects implementation:

o Servers.

- The object is implemented as a parallel process (server) with entry points corresponding to object operations. If no calls are made to it, the object suspends itself and waits for further requests for service

o Active objects

- Objects are implemented as parallel processes and the internal object state may be changed by the object itself and not simply by external calls.

Active transponder object

- Active objects may have their attributes modified by operations but may also update them autonomously using internal operations

- Transponder object broadcasts an aircraft's position. The position may be updated using a satellite positioning system. The object periodically updates the position by triangulation from satellites

5

Software Engineering Lecture Notes. Dr. David A. Oyemade

An active transponder object

Java threads

- Threads in Java are a simple construct for implementing concurrent objects
- Threads must include a method called run() and this is started up by the Java run-time system
- Active objects typically include an infinite loop so that they are always carrying out the computation

An object-oriented design process

- Define the context and modes of use of the system
- Design the system architecture
- Identify the principal system objects
- Develop design models
- Specify object interfaces

Use Cases – System behavior

- Use-cases are a scenario based technique which identify the actors in an interaction and which describe the interaction
- A set of use cases should describe all possible interactions with the system
- Other types of UML diagrams can add detail to use cases

Use cases

- Specify system behavior without specifying implementation
- Focus on interaction with users
- Validate understanding of requirements
- Decompose system into coherent categories of functionality
- Use cases and informal scenarios

6

Software Engineering Lecture Notes. Dr. David A. Oyemade

- More abstract than informal scenarios. Avoids reference to specific values
- A single use-case usually encompasses multiple scenarios

OBJECT IDENTIFICATION

There are different proposals:

1. Use a grammatical analysis of a natural language description of a system. Objects and attributes are nouns; operations or services are verbs.

2. Use tangible entities (things) in the application domain such as aircraft, roles such as manager, events such as request, interactions such as meetings, locations such as offices, organizational units such as companies, and so on.
3. Use a behavioral approach where the designer first understands the overall behavior of the system. Understanding is derived on who initiates and participates in these behaviors.
4. Use a scenario-based analysis where various scenarios of a system use are identified and analyzed in turn.

Use case format - LMS (Library Management System)

- Use Case: Check out resource

Precondition: The Library staff has successfully identified herself to the library system by entering a valid library staff identification and password. A library database, containing information concerning the library holdings and the patrons of the library, has been created and initialized.

Use case format - LMS

- Main flow of events: The use case starts when the Library staff requests the Check out resource function from the system main menu. The system then initiates the Validate patron use case. If that use case ends successfully and a valid patron id has been entered into the system, the system initiates the Enter resource use case. If the Enter resource use case ends successfully, the Determine due date use case is initiated. The Enter resource and Determine due date use-case-pair is executed over and over until the Library staff commits the entry by pressing the enter key. The system then displays a list of valid resources that have been entered along with due dates for each resource. This ends the Check out resource use case.

- Use case format -- LMS

- Exceptional flow of events: If the Validate patron use case does not end with a valid patron id being entered, the system cancels the entire transaction and this use case ends

7

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Exceptional flow of events: if the Enter resource use case does not end with a valid resource Dewey call number having been entered, the system displays an appropriate warning message and continues the Check out resource use case by skipping the Determine due date use case and initiating the next instance of the Enter resource use case.

- Use case format -- LMS

- Postcondition: If the Validate patron use case does not end with a valid patron id being entered, nothing in the entire system has changed when this use case ends. If a valid library patron id was entered, then the Patron object is updated with the

Dewey call numbers of the Resource objects that have been checked out along with their due dates. In addition, the Resource objects are updated to have a status of checked-out along with the library id of the Patron who check it out.

- Use Case: Validate Patron
- Use case: Validate patron
- Precondition: Patron needs some service
- Main flow of events: The use case starts when the system prompts the Library staff for the Patron's library id. The Library staff can either scan in a card or type in an id and then commit the entry by pressing the enter key. The system then checks to see if the id is valid. If it is valid the system displays the library record for Patron, thus ending this use case
- Use case - Validate patron
- Exceptional flow of events: If the Library staff enters an invalid library id, the system cancels the entire transaction and this use case ends
- Exceptional flow of events: If the Library staff enters a library id that has expired then the system cancels the entire transaction and this use case ends
- Exceptional flow of events: If the Patron has too many resources checked out, he or she may not check out any additional resources. A message that tallies and lists the resources currently held by Patron is displayed on the screen and this use case ends
- Use case - Validate patron
- Exceptional flow of events: If the Patron has any library resources that are more than two weeks overdue, then these resources are listed on the screen and this use case ends.

8

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Postcondition: The system retrieves and displays the library record of the Patron. The system's state has not changed.

Architectural design

- Once interactions between the system and its environment have been understood, you use this information for designing the system architecture
- There should be no more than 7 entities in an architectural model

Key points

- OOD is an approach to design so that design components have their own private state and operations
- Objects should have constructor and inspection operations. They provide services to other objects
- Objects may be implemented sequentially or concurrently

- The Unified Modeling Language provides different notations for defining different object models
- Key points
- A range of different models may be produced during an object-oriented design process. These include static and dynamic system models
- Object-oriented design simplifies system evolution

9

Software Engineering Lecture Notes. Dr. David A. Oyemade

4 Design with Reuse

- Building software from reusable components.
- In most engineering disciplines, systems are designed by composing existing components that have been used in other systems
- Software engineering has been more focused on original development but it is now recognised that to achieve better software, more quickly and at lower cost, we need to adopt a design process that is based on systematic reuse

Reuse-based software engineering

- Application system reuse
 - The whole of an application system may be reused either by incorporating it without change into other systems (COTS reuse) or by developing application families
- Component reuse
 - Components of an application from sub-systems to single objects may be reused
- Function reuse
 - Software components that implement a single well-defined function may be reused

Reuse practice

- Application system reuse

10

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Widely practised as software systems are implemented as application families. COTS reuse is becoming increasingly common
- Component reuse
 - Now seen as the key to effective and widespread reuse through component-based software engineering. However, it is still relatively immature
- Function reuse

- Common in some application domains (e.g. engineering) where domain-specific libraries of reusable functions have been established

Benefits of reuse

- Increased reliability
- Components exercised in working systems
- Reduced process risk
- Less uncertainty in development costs
- Effective use of specialists
- Reuse components instead of people
- Standards compliance
- Embed standards in reusable components
- Accelerated development
- Avoid original development and hence speed-up production

11

Software Engineering Lecture Notes. Dr. David A. Oyemade

Requirements for design with reuse

- It must be possible to find appropriate reusable components
- The re-user of the component must be confident that the components will be reliable and will behave as specified
- The components must be documented so that they can be understood and, where appropriate, modified

Reuse problems

- Increased maintenance costs
- Lack of tool support
- Not-invented-here syndrome
- Maintaining a component library
- Finding and adapting reusable components

Generator-based reuse

- Program generators involve the reuse of standard patterns and algorithms
- These are embedded in the generator and parameterised by user commands. A program is then automatically generated
- Generator-based reuse is possible when domain abstractions and their mapping to executable code can be identified
- A domain specific language is used to compose and control these abstractions

Types of program generator

- Application generators for business data processing

- Parser and lexical analyser generators for language processing
- Code generators in CASE tools

12

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Generator-based reuse is very cost-effective but its applicability is limited to a relatively small number of application domains
- It is easier for end-users to develop programs using generators compared to other component-based approaches to reuse

Reuse through program generation

Component-based development

- Component-based software engineering (CBSE) is an approach to software development that relies on reuse
- It emerged from the failure of object-oriented development to support effective reuse. Single object classes are too detailed and specific
- Components are more abstract than object classes and can be considered to be stand-alone service providers

Components

- Components provide a service without regard to where the component is executing or its programming language

13

Software Engineering Lecture Notes. Dr. David A. Oyemade

- A component is an independent executable entity that can be made up of one or more executable objects
- The component interface is published and all interactions are through the published interface
- Components can range in size from simple functions to entire application systems

Component interfaces

- Provides interface
- Defines the services that are provided by the component to other components
- Requires interface
- Defines the services that specifies what services must be made available for the component to execute as specified

Printing services component

14

Software Engineering Lecture Notes. Dr. David A. Oyemade

Component abstractions

- Functional abstraction
 - The component implements a single function such as a mathematical function
- Casual groupings
 - The component is a collection of loosely related entities that might be data declarations, functions, etc.
- Data abstractions
 - The component represents a data abstraction or class in an object-oriented language
- Cluster abstractions
 - The component is a group of related classes that work together
- System abstraction
 - The component is an entire self-contained system

CBSE processes

- Component-based development can be integrated into a standard software process by incorporating a reuse activity in the process

15

Software Engineering Lecture Notes. Dr. David A. Oyemade

- However, in reuse-driven development, the system requirements are modified to reflect the components that are available
- CBSE usually involves a prototyping or an incremental development process with components being 'glued together' using a scripting language

An opportunistic reuse process

Development with reuse

CBSE problems

- Component incompatibilities may mean that cost and schedule savings are less than expected
- Finding and understanding components
- Managing evolution as requirements change in situations where it may be impossible to change the system components

16

Software Engineering Lecture Notes. Dr. David A. Oyemade

Application frameworks

- Frameworks are a sub-system design made up of a collection of abstract and concrete classes and the interfaces between them

- The sub-system is implemented by adding components to fill in parts of the design and by instantiating the abstract classes in the framework
- Frameworks are moderately large entities that can be reused
- Framework classes
- System infrastructure frameworks
 - Support the development of system infrastructures such as communications, user interfaces and compilers
- Middleware integration frameworks
 - Standards and classes that support component communication and information exchange
- Enterprise application frameworks
 - Support the development of specific types of application such as telecommunications or financial systems

Extending frameworks

- Frameworks are generic and are extended to create a more specific application or sub-system
- Extending the framework involves
 - Adding concrete classes that inherit operations from abstract classes in the framework
 - Adding methods that are called in response to events that are recognised by the framework
- Problem with frameworks is their complexity and the time it takes to use them effectively
- Model-view controller

17

Software Engineering Lecture Notes. Dr. David A. Oyemade

- System infrastructure framework for GUI design
- Allows for multiple presentations of an object and separate interactions with these presentations

MVC framework involves the instantiation of a number of Model-view controller

COTS product reuse

- COTS - Commercial Off-The-Shelf systems
- COTS systems are usually complete application systems that offer an API (Application Programming Interface)
- Building large systems by integrating COTS systems is now a viable development strategy for some types of system such as E-commerce systems
- COTS system integration problems

- Lack of control over functionality and performance
 - COTS systems may be less effective than they appear
- Problems with COTS system inter-operability
 - Different COTS systems may make different assumptions that means integration is difficult
- No control over system evolution
 - COTS vendors not system users control evolution
- Support from COTS vendors
 - COTS vendors may not offer support over the lifetime of the product

Component development for reuse

- Components for reuse may be specially constructed by generalising existing components

18

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Component reusability
 - Should reflect stable domain abstractions
 - Should hide state representation
 - Should be as independent as possible
 - Should publish exceptions through the component interface

There is a trade-off between reusability and usability.

- The more general the interface, the greater the reusability but it is then more complex and hence less usable

Reusable components

- The development cost of reusable components is higher than the cost of specific equivalents. This extra reusability enhancement cost should be an organization rather than a project cost
- Generic components may be less space-efficient and may have longer execution times than their specific equivalents

Reusability enhancement

- Name generalisation
 - Names in a component may be modified so that they are not a direct reflection of a specific application entity
- Operation generalisation
 - Operations may be added to provide extra functionality and application specific operations may be removed

- Exception generalisation
- Application specific exceptions are removed and exception management added to increase the robustness of the component

19

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Component certification
- Component is certified as reusable

Reusability enhancement process

Application families

- An application family or product line is a related set of applications that has a common, domain-specific architecture
- The common core of the application family is reused each time a new application is required
- Each specific application is specialised in some way

Application family specialisation

- Platform specialisation
- Different versions of the application are developed for different platforms
- Configuration specialisation
- Different versions of the application are created to handle different peripheral devices
- Functional specialisation
- Different versions of the application are created for customers with different requirements

Inventory management systems

20

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Resource database
- Maintains details of the things that are being managed

Application family architectures

- Architectures must be structured in such a way as to separate different sub-systems and to allow them to be modified
- The architecture should also separate entities and their descriptions and the higher levels in the system access entities through descriptions rather than directly

Library system

- The resources being managed are the books in the library
- Additional domain-specific functionality (issue, borrow, etc.) must be added for

this application

Family member development

- Elicit stakeholder requirements
- Use existing family member as a prototype
- Choose closest-fit family member
- Find the family member that best meets the requirements
- Re-negotiate requirements
- Adapt requirements as necessary to capabilities of the software

21

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Adapt existing system
- Develop new modules and make changes for family member
- Deliver new family member
- Document key features for further member development

Design patterns

- A design pattern is a way of reusing abstract knowledge about a problem and its solution
- A pattern is a description of the problem and the essence of its solution
- It should be sufficiently abstract to be reused in different settings
- Patterns often rely on object characteristics such as inheritance and polymorphism
- Pattern elements
- Name
- A meaningful pattern identifier
- Problem description
- Solution description
- Not a concrete design but a template for a design solution that can be instantiated in different ways
- Consequences
- The results and trade-offs of applying the pattern
- Multiple displays
- Key points

22

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Design with reuse involves designing software around good design and existing components

- Advantages are lower costs, faster software development and lower risks
- Component-based software engineering relies on black-box components with defined requires and provides interfaces
- COTS product reuse is concerned with the reuse of large, off-the-shelf systems

Key points

- Software components for reuse should be independent, should reflect stable domain abstractions and should provide access to state through interface operations
- Application families are related applications developed around a common core
- Design patterns are high-level abstractions that document successful design solutions

23

Software Engineering Lecture Notes. Dr. David A. Oyemade

Assignment: Model your final year project using object oriented analysis and design (OOAD). Use CASE tools for the modeling and an object oriented programming language such as Java, C# or C++ for the implementation.

(30/6/2016).

24

- > The Pipe and Filter is an architectural design pattern that allows for stream/asynchronous processing
- > Components are referred to as filters, and connectors between the filters are called pipes
- > Each filter is responsible for applying a function to the given data; this is known as filtering
- > The final output is given to the consumer, known as a sink.

Applicability

- > Best for large processes that can be broken down into multiple steps. This way, each filter will be responsible for one of the steps and they can run simultaneously to produce all the data needed for the final outcome.

Example

- > UNIX pipe command. Output from one function as input to another function

An Implementation

- > A demonstration of how a pizza is made using this style. We are using 5 filters, with 3 of them working asynchronously. We implement the first few filters to process the raw ingredients to create the basic elements for the pizza (the vegetables, the sauce, and the pizza base). When all three of these have been completed, we can assemble them. After it has been assembled, we can then add the cheese to the pizza, and bake it, before delivering it to our customer, the sink.

Software Design Patterns

MVC Pattern

MVC Pattern stands for Model-View-Controller Pattern. This pattern is used to separate application's concerns.

- Model - Model represents an object or JAVA POJO carrying data. It can also have logic to update controller if its data changes.
- View - View represents the visualization of the data that model contains.
- Controller - Controller acts on both model and view. It controls the data flow into model object and updates the view whenever data changes. It keeps view and model separate.

Implementation

We are going to create a Student object acting as a model. StudentView will be a view class which can print student details on console and StudentController is the controller class responsible to store data in Student object and update view StudentView accordingly.

MVCPatternDemo, our demo class, will use StudentController to demonstrate use of MVC pattern.

Step 1

Create Model.

Student.java

```
public class Student {  
    private String rollNo;  
    private String name;  
  
    public String getRollNo() {  
        return rollNo;  
    }  
  
    public void setRollNo(String rollNo) {  
        this.rollNo = rollNo;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Step 2

Create View.

StudentView.java

```
public class StudentView {  
    public void printStudentDetails(String studentName, String  
studentRollNo){  
        System.out.println("Student: ");  
        System.out.println("Name: " + studentName);  
        System.out.println("Roll No: " + studentRollNo);  
    }  
}
```

Step 3

Create Controller.

StudentController.java

```
public class StudentController {
    private Student model;
    private StudentView view;

    public StudentController(Student model, StudentView view){
        this.model = model;
        this.view = view;
    }

    public void setStudentName(String name){
        model.setName(name);
    }

    public String getStudentName(){
        return model.getName();
    }

    public void setStudentRollNo(String rollNo){
        model.setRollNo(rollNo);
    }

    public String getStudentRollNo(){
        return model.getRollNo();
    }

    public void updateView(){
        view.printStudentDetails(model.getName(), model.getRollNo());
    }
}
```

Step 4

Use the StudentController methods to demonstrate MVC design pattern usage.

MVCPatternDemo.java

```
public class MVCPatternDemo {
    public static void main(String[] args) {

        //fetch student record based on his roll no from the database
        Student model = retrieveStudentFromDatabase();

        //Create a view : to write student details on console
        StudentView view = new StudentView();

        StudentController controller = new StudentController(model,
            view);

        controller.updateView();

        //update model data
        controller.setStudentName("John");

        controller.updateView();

        private static Student retrieveStudentFromDatabase(){
            Student student = new Student();
            student.setName("Robert");
            student.setRollNo("10");
            return student;
        }
    }
}
```

Step 5

Verify the output.

Student:

Name: Robert

Roll No: 10

Student:

Name: John

Roll No: 10

Observer Pattern

Observer pattern is used when there is one-to-many relationship between objects such as if one object is modified, its dependent objects are to be notified automatically.

Observer pattern falls under behavioral pattern category.

Implementation

Observer pattern uses three actor classes. Subject, Observer and Client. Subject is an object having methods to attach and detach observers to a client object. We have created an abstract class Observer and a concrete class Subject that is extending class Observer.

ObserverPatternDemo, our demo class, will use Subject and concrete class object to show observer pattern in action.

Step 1

Create Subject class.

Subject.java

```
import java.util.ArrayList;
import java.util.List;

public class Subject {

    private List<Observer> observers = new ArrayList<Observer>();
    private int state;

    public int getState() {
        return state;
    }

    public void setState(int state) {
        this.state = state;
        notifyAllObservers();
    }

    public void attach(Observer observer){
        observers.add(observer);
    }

    public void notifyAllObservers(){
        for (Observer observer : observers) {
            observer.update();
        }
    }
}
```

Step 2

Create Observer class.

Observer.java

```
public abstract class Observer {  
    protected Subject subject;  
    public abstract void update();  
}
```

Step 3

Create concrete observer classes

BinaryObserver.java

```
public class BinaryObserver extends Observer{  
  
    public BinaryObserver(Subject subject){  
        this.subject = subject;  
        this.subject.attach(this);  
    }  
  
    @Override  
    public void update() {  
        System.out.println( "Binary String: " +  
            Integer.toBinaryString( subject.getState() ) );  
    }  
}
```

OctalObserver.java

```
public class OctalObserver extends Observer{  
  
    public OctalObserver(Subject subject){  
        this.subject = subject;  
        this.subject.attach(this);  
    }  
  
    @Override  
    public void update() {  
        System.out.println( "Octal String: " + Integer.toOctalString(  
            subject.getState() ) );  
    }  
}
```

HexaObserver.java

```
public class HexaObserver extends Observer{  
  
    public HexaObserver(Subject subject){  
        this.subject = subject;  
        this.subject.attach(this);  
    }  
  
    @Override  
    public void update() {  
        System.out.println( "Hex String: " + Integer.toHexString(  
            subject.getState() ).toUpperCase() );  
    }  
}
```

HexaObserver.java

```
public class HexaObserver extends Observer{  
  
    public HexaObserver(Subject subject){  
        this.subject = subject;  
        this.subject.attach(this);  
    }  
  
    @Override  
    public void update() {  
        System.out.println( "Hex String: " + Integer.toHexString(  
            subject.getState() ).toUpperCase() );  
    }  
}
```

Step 4

Use Subject and concrete observer objects.

ObserverPatternDemo.java

```
public class ObserverPatternDemo {  
    public static void main(String[] args) {  
        Subject subject = new Subject();  
    }  
}
```

```

new HexaObserver(subject);
new OctalObserver(subject);
new BinaryObserver(subject);

System.out.println("First state change: 15");
subject.setState(15);
System.out.println("Second state change: 10");
subject.setState(10);

```

Step 5

Verify the output.

First state change: 15

Hex String: F

Octal String: 17

Binary String: 1111

Second state change: 10

Hex String: A

Octal String: 12

Binary String: 1010

Factory Method Pattern

A Factory Pattern or Factory Method Pattern says that just define an interface or abstract class for creating an object but let the subclasses decide which class to instantiate. In other words, subclasses are responsible to create the instance of the class.

The Factory Method Pattern is also known as Virtual Constructor.

Advantage of Factory Design Pattern

- o Factory Method Pattern allows the sub-classes to choose the type of objects to create.
- o It promotes the loose-coupling by eliminating the need to bind application-specific classes into the code. That means the code interacts solely with the resultant interface or abstract class, so that it will work with any classes that implement that interface or that extends that abstract class.

Usage of Factory Design Pattern

- o When a class doesn't know what sub-classes will be required to create
- o When a class wants that its sub-classes specify the objects to be created.
- o When the parent classes choose the creation of objects to its sub-classes.

UML for Factory Method Pattern

- o We are going to create a Plan abstract class and concrete classes that extends the Plan abstract class. A factory class GetPlanFactory is defined as a next step.
- o GenerateBill class will use GetPlanFactory to get a Plan object. It will pass information (DOMESTICPLAN / COMMERCIALPLAN / INSTITUTIONALPLAN) to

GetPlanFactory to get the type of object it needs.

Calculate Electricity Bill : A Real World Example of Factory Method

Step 1: Create a Plan abstract class

```
import java.io.*;
abstract class Plan{
protected double rate;
abstract void getRate();
public void calculateBill(int units){
System.out.println(units*rate);
} //end of Plan class.
```

Step 2: Create the concrete classes that extends Plan abstract class.

```
class DomesticPlan extends Plan{
//@override
public void getRate(){
rate=3.50;
} //end of DomesticPlan class.
class CommercialPlan extends Plan{
//@override
public void getRate(){
rate=7.50;
} //end of CommercialPlan class.
class InstitutionalPlan extends Plan{
//@override
public void getRate(){
rate=5.50;
} //end of InstitutionalPlan class.
```

Step 3: Create a GetPlanFactory to generate object of concrete classes based on given information..

```
class GetPlanFactory{
//use getPlan method to get object of type Plan
public Plan getPlan(String planType){
if(planType == null){
return null;
}
if(planType.equalsIgnoreCase("DOMESTICPLAN")) {
return new DomesticPlan();
}
else if(planType.equalsIgnoreCase("COMMERCIALPLAN")){
return new CommercialPlan();
}
else if(planType.equalsIgnoreCase("INSTITUTIONALPLAN")) {
return new InstitutionalPlan();
}
```

```
return null;
} //end of GetPlanFactory class.
```

Step 4: Generate Bill by using the GetPlanFactory to get the object of concrete classes by passing an information such as type of plan DOMESTICPLAN or COMMERCIALPLAN or INSTITUTIONALPLAN.

```
import java.io.*;
class GenerateBill{
public static void main(String args[])throws IOException{
GetPlanFactory planFactory = new GetPlanFactory();

System.out.print("Enter the name of plan for which the bill will be generated: ");
BufferedReader br=new BufferedReader(new InputStreamReader(System.in));

String planName=br.readLine();
System.out.print("Enter the number of units for bill will be calculated: ");
int units=Integer.parseInt(br.readLine());

Plan p = planFactory.getPlan(planName);
//call getRate() method and calculateBill()method of DomesticPaln.

System.out.print("Bill amount for "+planName+" of "+units+" units is: ");
p.getRate();
p.calculateBill(units);
} //end of GenerateBill class.
```

download this Electricity bill Example

Output

References:

https://www.tutorialspoint.com/design_pattern/mvc_pattern.htm

https://www.tutorialspoint.com/design_pattern/observer_pattern.htm

[https://www.javatpoint.com/factory-method-design-](https://www.javatpoint.com/factory-method-design-pattern#:~:text=A%20Factory%20Pattern%20or%20Factory,the%20instance%20of%20the%20class.)

[pattern#:~:text=A%20Factory%20Pattern%20or%20Factory,the%20instance%20of%20the%20class.](https://www.javatpoint.com/factory-method-design-pattern#:~:text=A%20Factory%20Pattern%20or%20Factory,the%20instance%20of%20the%20class.)

Paper 7 of 9 — CSC-411-LECTURE-NOTES-1

Level 400 • First Semester • Handout

Software Engineering Lecture Notes. Dr. David A. Oyemade

SOFTWARE ENGINEERING

LECTURE NOTES

Dr. David A. Oyemade

1

Software Engineering Lecture Notes. Dr. David A. Oyemade

CSC 411: SOFTWARE ENGINEERING: (3 Units)

Objectives/Learning Outcomes

At the end of this course you will understand the following aspects of Software Engineering and you will be able to apply them to practical software projects:

Software Design: Software architecture

Design Patterns

Object Oriented Analysis & Design

Design for re-use.

Using APIs: API programming Class browsers and related tools

Component based computing.

Software tools and Environment:

Requirements analysis and design

Software Modeling Tools,

Testing tools, Tool integration mechanisms.

2

Software Engineering Lecture Notes. Dr. David A. Oyemade

1. Introduction

Getting started with software engineering

Objectives

To introduce software engineering and to explain its importance

T

o set out the answers to key questions about software engineering

T

o introduce ethical and professional issues and to explain why they are of concern to software engineers

Topics covered

FAQs about software engineering

Software engineering

The economies of ALL developed nations are

dependent on software

M

ore and more systems are software controlled

S

oftware engineering is concerned with theories, methods and tools for professional software development

3

Software Engineering Lecture Notes. Dr. David A. Oyemade

Software engineering expenditure represents a significant fraction of GNP in all developed countries

FAQs about software engineering

What is software?

W

hat is software engineering?

W

hat is the difference between software engineering and computer science?

W

hat is the difference between software engineering and system engineering?

W

hat is a software process?

W

hat is a software process model?

FAQs about software engineering

What are the costs of software engineering?

W

hat are software engineering methods?

W

hat is CASE (Computer-Aided Software Engineering)

W

hat are the attributes of good software?

W

hat are the key challenges facing software engineering?

What is software?

Computer programs and associated documentation

S

oftware products may be developed for a particular customer or may be developed for a general market

S

oftware products may be

- Generic - developed to be sold to a range of different customers
- Custom - developed for a single customer according to their specification

What is software engineering?

Software engineering is an engineering discipline which is concerned with all aspects of software production

S

oftware engineers should adopt a systematic and organised approach to their work and use appropriate tools and techniques depending on the problem to be solved, the development constraints and the resources available

What is the difference between software engineering and computer science?

4

Software Engineering Lecture Notes. Dr. David A. Oyemade

Computer science is concerned with theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software

C

omputer science theories are currently insufficient to act as a complete underpinning for software engineering

What is a software process?

A set of activities whose goal is the development or evolution of software

G

eneric activities in all software processes are:

- Specification - what the system should do and its development constraints
- Development - production of the software system
- Validation - checking that the software is what the customer wants
- Evolution - changing the software in response to changing demands

What is a software process model?

A simplified representation of a software process, presented from a specific perspective

E

xamples of process perspectives are

- Workflow perspective - sequence of activities
- Data-flow perspective - information flow
- Role/action perspective - who does what

Generic process models

- Waterfall

- Evolutionary development
- Formal transformation
- Integration from reusable components

What are software engineering methods?

Structured approaches to software development which include system models, notations, rules, design advice and process guidance

Model descriptions

- Descriptions of graphical models which should be produced

Rules

- Constraints applied to system models

Recommendations

- Advice on good design practice

Process guidance

- What activities to follow

What is CASE (Computer-Aided Software Engineering)

5

Software Engineering Lecture Notes. Dr. David A. Oyemade

Software systems which are intended to provide automated support for software process activities. CASE systems are often used for method support

U

pper-CASE

- Tools to support the early process activities of requirements and design

Lower-CASE

- Tools to support later activities such as programming, debugging and testing

What are the attributes of good software?

The software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable

M

aintainability

- Software must evolve to meet changing needs

Dependability

- Software must be trustworthy

Efficiency

- Software should not make wasteful use of system resources

Usability

- Software must be usable by the users for which it was designed

What are the key challenges facing software engineering?

Coping with legacy systems, coping with increasing diversity and coping with demands for reduced delivery times

L

Legacy systems

- Old, valuable systems must be maintained and updated

Heterogeneity

- Systems are distributed and include a mix of hardware and software

Delivery

- There is increasing pressure for faster delivery of software

Key points

Software engineering is an engineering discipline which is concerned with all aspects of software production.

Software products consist of developed programs and associated documentation. Essential product attributes are maintainability, dependability, efficiency and usability.

The software process consists of activities which are involved in developing software products.

Basic activities are software specification, development, validation and evolution.

Methods are organised ways of producing software. They include suggestions for the process to be

M

followed, the notations to be used, rules governing the system descriptions which are produced and design

guidelines.

C

ASE tools are software systems which are designed to support routine activities in the software process such as editing design diagrams, checking diagram consistency and keeping track of program tests which have been run.

6

Software Engineering Lecture Notes. Dr. David A. Oyemade

2. Software Processes

Informal Scenarios

Help understand the problem

C

Come up with questions on requirements

H

Help constrain architecture

Software Processes

Coherent sets of activities for specifying, designing,

implementing and testing software systems

Objectives

To introduce software process models

T

To describe a number of different process models and when they may be

used

o describe process models for requirements engineering, software

T

development, testing and evolution

Topics covered

Software process models

P

rocess iteration

S

oftware specification

oftware design and implementation

S

S

oftware validation

S

oftware evolution

A

utomated process support

7

Software Engineering Lecture Notes. Dr. David A. Oyemade

The software process

A structured set of activities required to develop a

software system

- Specification
- Design
- Validation
- Evolution

A software process model is an abstract representation of a process. It

presents a description of a process from some particular perspective

Generic software process models

The waterfall model

- Separate and distinct phases of specification and development

Evolutionary development

- Specification and development are interleaved

Formal systems development

- A mathematical system model is formally transformed to an implementation

Reuse-based development

- The system is assembled from existing components

Waterfall model

8

Software Engineering Lecture Notes. Dr. David A. Oyemade

Waterfall model phases

Requirements analysis and definition

S

ystem and software design

I mplementation and unit testing

I ntegration and system testing

O

peration and maintenance

T

he drawback of the waterfall model is the difficulty of accommodating

change after the process is underway

Waterfall model problems

Inflexible partitioning of the project into distinct stages

T

his makes it difficult to respond to changing customer requirements

T

herefore, this model is only appropriate when the requirements are

well-understood

Evolutionary development

Exploratory development

- Objective is to work with customers and to evolve a final system from an initial outline specification. Should start with well-understood requirements

Throw-away prototyping

- Objective is to understand the system requirements. Should start with poorly understood requirements

Evolutionary development

9

Software Engineering Lecture Notes. Dr. David A. Oyemade

Evolutionary development

Problems

- Lack of process visibility
- Systems are often poorly structured
- Special skills (e.g. in languages for rapid prototyping) may be required

Applicability

- For small or medium-size interactive systems
- For parts of large systems (e.g. the user interface)
- For short-lifetime systems

Formal systems development

Based on the transformation of a mathematical specification through different representations to an executable program

T

ransformations are 'correctness-preserving' so it is straightforward to show that the program conforms to its specification

E

mbodied in the 'Cleanroom' approach to software development

Formal systems development

10

Software Engineering Lecture Notes. Dr. David A. Oyemade

Reuse-oriented development

Based on systematic reuse where systems are integrated from existing components or COTS (Commercial-off-the-shelf) systems

P

rocess stages

- Component analysis
- Requirements modification
- System design with reuse
- Development and integration

This approach is becoming more important but still limited experience with it

Reuse-oriented development

Process iteration

System requirements ALWAYS evolve in the course of a project so process iteration where earlier stages are reworked is always part of the process for large systems

Iteration can be applied to any of the generic process models

T

wo (related) approaches

- Incremental development
- Spiral development

Incremental development

Rather than deliver the system as a single delivery, the development and delivery is broken down into increments with each increment delivering part of the required functionality

U

ser requirements are prioritised and the highest priority requirements are included in early increments

O

nce the development of an increment is started, the requirements are frozen
though requirements for later increments can continue to evolve

11

Software Engineering Lecture Notes. Dr. David A. Oyemade

Incremental development

Incremental development advantages

Customer value can be delivered with each increment so system

functionality is available earlier

E

arly increments act as a prototype to help elicit requirements for later
increments

L

ower risk of overall project failure

T

he highest priority system services tend to receive the most testing

Extreme programming

New approach to development based on the development and delivery of

very small increments of functionality

elies on constant code improvement, user involvement in the development

R

team and pairwise programming

Software specification

The process of establishing what services are required and the constraints on
the system's operation and development

R

equirements engineering process

- Feasibility study
- Requirements elicitation and analysis
- Requirements specification
- Requirements validation

12

Software Engineering Lecture Notes. Dr. David A. Oyemade

The requirements engineering process

Software design and implementation

The process of converting the system specification into an executable system

S

oftware design

- Design a software structure that realises the specification

Implementation

- Translate this structure into an executable program

The activities of design and implementation are closely related and may be inter-leaved

Design process activities

Architectural design

A

bstract specification

I nterface design

C

omponent design

D

ata structure design

A

lgorithm design

The software design process

13

Software Engineering Lecture Notes. Dr. David A. Oyemade

Design methods

Systematic approaches to developing a software design

T

he design is usually documented as a set of graphical models

P

ossible models

- Data-flow model
- Entity-relation-attribute model
- Structural model
- Object models

Programming and debugging

Translating a design into a program and removing errors from that program

P

rogramming is a personal activity - there is no generic programming

process

P

rogrammers carry out some program testing to discover faults in the

program and remove these faults in the debugging process

The debugging process

Software validation

14

Software Engineering Lecture Notes. Dr. David A. Oyemade

Verification and validation is intended to show that a system conforms to its specification and meets the requirements of the system customer

Involves checking and review processes and system testing

System

testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system

The testing process

Testing stages

Unit testing

- Individual components are tested

Module testing

- Related collections of dependent components are tested

Sub-system testing

- Modules are integrated into sub-systems and tested. The focus here should be on interface testing

System testing

- Testing of the system as a whole. Testing of emergent properties

Acceptance testing

- Testing with customer data to check that it is acceptable

15

Software Engineering Lecture Notes. Dr. David A. Oyemade

Testing phases

Software evolution

Software is inherently flexible and can change.

As

requirements change through changing business circumstances, the software that supports the business must also evolve and change

Although

there has been a demarcation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new

System evolution

Automated process support (CASE)

16

Software Engineering Lecture Notes. Dr. David A. Oyemade

Computer-aided software engineering (CASE) is software to support software development and evolution processes

A

ctivity automation

- Graphical editors for system model development
- Data dictionary to manage design entities
- Graphical UI builder for user interface construction
- Debuggers to support program fault finding
- Automated translators to generate new versions of a program

Key points

Software processes are the activities involved in producing and evolving a software system. They are represented in a software process model
 eneral activities are specification, design and implementation, validation

G

and evolution

G

eneric process models describe the organisation of software processes

I terative process models describe the software process as a cycle of activities

equirements engineering is the process of developing a software

R

specification

esign and implementation processes transform the specification to an

D

executable program

V

alidation involves checking that the system meets to its specification and

user needs

volution is concerned with modifying the system after it is in use

E

ASE technology supports software process activities

C

17

Software Engineering Lecture Notes. Dr. David A. Oyemade

18

Paper 8 of 9 — CSC-411-LECTURE-NOTES-2

Level 400 • First Semester • Handout

Software Engineering Lecture Notes. Dr. David A. Oyemade

2 Design with Reuse

- Building software from reusable components.
- In most engineering disciplines, systems are designed by composing existing components that have been used in other systems
- Software engineering has been more focused on original development but it is now recognised that to achieve better software, more quickly and at lower cost, we need to adopt a design process that is based on systematic reuse

Reuse-based software engineering

- Application system reuse
 - The whole of an application system may be reused either by incorporating it without change into other systems (COTS reuse) or by developing application families
- Component reuse
 - Components of an application from sub-systems to single objects may be reused
- Function reuse
 - Software components that implement a single well-defined function may be reused

Reuse practice

- Application system reuse
 - Widely practised as software systems are implemented as application families. COTS reuse is becoming increasingly common

1

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Component reuse
 - Now seen as the key to effective and widespread reuse through component-based software engineering. However, it is still relatively immature
- Function reuse
 - Common in some application domains (e.g. engineering) where domain-specific libraries of reusable functions have been established

Benefits of reuse

- Increased reliability

- Components exercised in working systems
- Reduced process risk
- Less uncertainty in development costs
- Effective use of specialists
- Reuse components instead of people
- Standards compliance
- Embed standards in reusable components
- Accelerated development
- Avoid original development and hence speed-up production

Requirements for design with reuse

2

Software Engineering Lecture Notes. Dr. David A. Oyemade

- It must be possible to find appropriate reusable components
- The re-user of the component must be confident that the components will be reliable and will behave as specified
- The components must be documented so that they can be understood and, where appropriate, modified

Reuse problems

- Increased maintenance costs
- Lack of tool support
- Not-invented-here syndrome
- Maintaining a component library
- Finding and adapting reusable components

Generator-based reuse

- Program generators involve the reuse of standard patterns and algorithms
- These are embedded in the generator and parameterised by user commands. A program is then automatically generated
- Generator-based reuse is possible when domain abstractions and their mapping to executable code can be identified
- A domain specific language is used to compose and control these abstractions

Types of program generator

- Application generators for business data processing
- Parser and lexical analyser generators for language processing
- Code generators in CASE tools

3

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Generator-based reuse is very cost-effective but its applicability is limited to a relatively small number of application domains
- It is easier for end-users to develop programs using generators compared to other component-based approaches to reuse

Reuse through program generation

Component-based development

- Component-based software engineering (CBSE) is an approach to software development that relies on reuse
- It emerged from the failure of object-oriented development to support effective reuse. Single object classes are too detailed and specific
- Components are more abstract than object classes and can be considered to be stand-alone service providers

Components

- Components provide a service without regard to where the component is executing or its programming language

4

Software Engineering Lecture Notes. Dr. David A. Oyemade

- A component is an independent executable entity that can be made up of one or more executable objects
- The component interface is published and all interactions are through the published interface
- Components can range in size from simple functions to entire application systems

Component interfaces

- Provides interface
- Defines the services that are provided by the component to other components
- Requires interface
- Defines the services that specifies what services must be made available for the component to execute as specified

Printing services component

5

Software Engineering Lecture Notes. Dr. David A. Oyemade

Component abstractions

- Functional abstraction
- The component implements a single function such as a mathematical function

- Casual groupings
 - The component is a collection of loosely related entities that might be data declarations, functions, etc.
- Data abstractions
 - The component represents a data abstraction or class in an object-oriented language
- Cluster abstractions
 - The component is a group of related classes that work together
- System abstraction
 - The component is an entire self-contained system

CBSE processes

- Component-based development can be integrated into a standard software process by incorporating a reuse activity in the process

6

Software Engineering Lecture Notes. Dr. David A. Oyemade

- However, in reuse-driven development, the system requirements are modified to reflect the components that are available
- CBSE usually involves a prototyping or an incremental development process with components being 'glued together' using a scripting language

An opportunistic reuse process

Development with reuse

CBSE problems

- Component incompatibilities may mean that cost and schedule savings are less than expected
- Finding and understanding components
- Managing evolution as requirements change in situations where it may be impossible to change the system components

7

Software Engineering Lecture Notes. Dr. David A. Oyemade

Application frameworks

- Frameworks are a sub-system design made up of a collection of abstract and concrete classes and the interfaces between them
- The sub-system is implemented by adding components to fill in parts of the design and by instantiating the abstract classes in the framework
- Frameworks are moderately large entities that can be reused
- Framework classes

- System infrastructure frameworks
 - Support the development of system infrastructures such as communications, user interfaces and compilers
- Middleware integration frameworks
 - Standards and classes that support component communication and information exchange
- Enterprise application frameworks
 - Support the development of specific types of application such as telecommunications or financial systems

Extending frameworks

- Frameworks are generic and are extended to create a more specific application or sub-system
- Extending the framework involves
 - Adding concrete classes that inherit operations from abstract classes in the framework
 - Adding methods that are called in response to events that are recognised by the framework
- Problem with frameworks is their complexity and the time it takes to use them effectively

8

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Model-view controller
- System infrastructure framework for GUI design
- Allows for multiple presentations of an object and separate interactions with these presentations

MVC framework involves the instantiation of a number of Model-view controller

COTS product reuse

- COTS - Commercial Off-The-Shelf systems
- COTS systems are usually complete application systems that offer an API (Application Programming Interface)
- Building large systems by integrating COTS systems is now a viable development strategy for some types of system such as E-commerce systems
- COTS system integration problems
 - Lack of control over functionality and performance
 - COTS systems may be less effective than they appear
- Problems with COTS system inter-operability

- Different COTS systems may make different assumptions that means integration is difficult

- No control over system evolution

- COTS vendors not system users control evolution

- Support from COTS vendors

- COTS vendors may not offer support over the lifetime of the product

Component development for reuse

9

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Components for reuse may be specially constructed by generalising existing components

- Component reusability

- Should reflect stable domain abstractions

- Should hide state representation

- Should be as independent as possible

- Should publish exceptions through the component interface

There is a trade-off between reusability and usability.

- The more general the interface, the greater the reusability but it is then more complex and hence less usable

Reusable components

- The development cost of reusable components is higher than the cost of specific equivalents. This extra reusability enhancement cost should be an organization rather than a project cost

- Generic components may be less

space-efficient and may have longer execution times than their specific equivalents

Reusability enhancement

- Name generalisation

- Names in a component may be modified so that they are not a direct reflection of a specific application entity

- Operation generalisation

- Operations may be added to provide extra functionality and application specific operations may be removed

- Exception generalisation

10

Software Engineering Lecture Notes. Dr. David A. Oyemade

– Application specific exceptions are removed and exception management added to increase the robustness of the component

- Component certification

– Component is certified as reusable

Reusability enhancement process

Design patterns

- A design pattern is a way of reusing abstract knowledge about a problem and its solution

- A pattern is a description of the problem and the essence of its solution

- It should be sufficiently abstract to be reused in different settings

- Patterns often rely on object characteristics such as inheritance and polymorphism

- Pattern elements

- Name

– A meaningful pattern identifier

- Problem description

- Solution description

– Not a concrete design but a template for a design solution that can be instantiated in different ways

11

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Consequences

– The results and trade-offs of applying the pattern

- Multiple displays

- Key points

- Design with reuse involves designing software around good design and existing components

- Advantages are lower costs, faster software development and lower risks

- Component-based software engineering relies on black-box components with defined requires and provides interfaces

- COTS product reuse is concerned with the reuse of large, off-the-shelf systems

Key points

- Software components for reuse should be independent, should reflect stable domain abstractions and should provide access to state through interface operations

- Application families are related applications developed around a common core

- Design patterns are high-level abstractions that document successful design

solutions

12

Software Engineering Lecture Notes. Dr. David A. Oyemade

Assignment: Model your final year project using object oriented analysis and design (OOAD). Use CASE tools for the modeling and an object oriented programming language such as Java, C# or C++ for the implementation.

(30/6/2016).

13

Paper 9 of 9 — CSC-411-LECTURE-NOTES-3

Level 400 • First Semester • Handout

Software Engineering Lecture Notes. Dr. David A. Oyemade

3 OBJECT ORIENTED DESIGN

Characteristics of OOD

Objects are abstractions of real-world or system entities and manage themselves

- Objects are independent and encapsulate state and representation information.
- System functionality is expressed in terms of object services
- Shared data areas are eliminated. Objects communicate by message passing

- Objects may be distributed and may execute sequentially or in parallel

Advantages of OOD

- Easier maintenance. Objects may be understood as stand-alone entities
- Objects are appropriate reusable components
- For some systems, there may be an obvious mapping from real world entities to system objects

Object-oriented development

- Object-oriented analysis, design and programming are related but distinct
- OOA is concerned with developing an object model of the application domain
- OOD is concerned with developing an object-oriented system model to implement requirements
- OOP is concerned with realising an OOD using an OO programming language such as Java or C++

Objects and object classes

- Objects are entities in a software system which represent instances of real-world and system entities
- Object classes are templates for objects. They may be used to create objects
- Object classes may inherit attributes and services from other object classes

1

Software Engineering Lecture Notes. Dr. David A. Oyemade

Objects

The Unified Modeling Language

- Several different notations for describing object-oriented designs were proposed in the 1980s and 1990s

- The Unified Modeling Language is an integration of these notations
- It describes notations for a number of different models that may be produced during OO analysis and design
- It is now a de facto standard for OO modelling

Employee object class (UML)

Object communication

- Conceptually, objects communicate by message passing.

Messages

- The name of the service requested by the calling object.
- Copies of the information required to execute the service and the name of a holder for the result of the service.

2

Software Engineering Lecture Notes. Dr. David A. Oyemade

o In practice, messages are often implemented by procedure calls

- Name = procedure name.
- Information = parameter list.

Message examples

// Call a method associated with a buffer

// object that returns the next value

// in the buffer

v = circularBuffer.Get () ;

// Call the method associated with a

// thermostat object that sets the

// temperature to be maintained

thermostat.setTemp (20) ;

Generalisation and inheritance

- Objects are members of classes which define attribute types and operations
- Classes may be arranged in a class hierarchy where one class (a super-class) is a generalisation of one or more other classes (sub-classes)
- A sub-class inherits the attributes and operations from its super class and may add new methods or attributes of its own
- Generalisation in the UML is implemented as inheritance in OO programming languages

3

Software Engineering Lecture Notes. Dr. David A. Oyemade

A generalisation hierarchy

Advantages of inheritance

- It is an abstraction mechanism which may be used to classify entities
- It is a reuse mechanism at both the design and the programming level
- The inheritance graph is a source of organisational knowledge about domains and systems

UML associations

- Objects and object classes participate in relationships with other objects and object classes
- In the UML, a generalised relationship is indicated by an association
- Associations may be annotated with information that describes the association
- Associations are general but may indicate that an attribute of an object is an associated object or that a method relies on an associated object

4

Software Engineering Lecture Notes. Dr. David A. Oyemade

An association model

Concurrent objects

- The nature of objects as self-contained entities make them suitable for concurrent implementation
- The message-passing model of object communication can be implemented directly if objects are running on separate processors in a distributed system

Servers and active objects

There are two kinds of concurrent objects implementation:

o Servers.

- The object is implemented as a parallel process (server) with entry points corresponding to object operations. If no calls are made to it, the object suspends itself and waits for further requests for service

o Active objects

- Objects are implemented as parallel processes and the internal object state may be changed by the object itself and not simply by external calls.

Active transponder object

- Active objects may have their attributes modified by operations but may also update them autonomously using internal operations
- Transponder object broadcasts an aircraft's position. The position may be updated using a satellite positioning system. The object periodically updates the position

by triangulation from satellites

5

Software Engineering Lecture Notes. Dr. David A. Oyemade

An active transponder object

Java threads

- Threads in Java are a simple construct for implementing concurrent objects
- Threads must include a method called run() and this is started up by the Java run-time system
- Active objects typically include an infinite loop so that they are always carrying out the computation

An object-oriented design process

- Define the context and modes of use of the system
- Design the system architecture
- Identify the principal system objects
- Develop design models
- Specify object interfaces

Use Cases – System behavior

- Use-cases are a scenario based technique which identify the actors in an interaction and which describe the interaction
- A set of use cases should describe all possible interactions with the system
- Other types of UML diagrams can add detail to use cases

Use cases

- Specify system behavior without specifying implementation
- Focus on interaction with users
- Validate understanding of requirements
- Decompose system into coherent categories of functionality
- Use cases and informal scenarios

6

Software Engineering Lecture Notes. Dr. David A. Oyemade

- More abstract than informal scenarios. Avoids reference to specific values
- A single use-case usually encompasses multiple scenarios

OBJECT IDENTIFICATION

There are different proposals:

1. Use a grammatical analysis of a natural language description of a system. Objects and attributes are nouns; operations or services are verbs.
2. Use tangible entities (things) in the application domain such as aircraft, roles such as manager, events such as request, interactions such as meetings, locations such

as offices, organizational units such as companies, and so on.

3. Use a behavioral approach where the designer first understands the overall behavior of the system. Understanding is derived on who initiates and participates in these behaviors.

4. Use a scenario-based analysis where various scenarios of a system use are identified and analyzed in turn.

Use case format - LMS (Library Management System)

- Use Case: Check out resource

Precondition: The Library staff has successfully identified herself to the library system by entering a valid library staff identification and password. A library database, containing information concerning the library holdings and the patrons of the library, has been created and initialized.

Use case format - LMS

- Main flow of events: The use case starts when the Library staff requests the Check out resource function from the system main menu. The system then initiates the Validate patron use case. If that use case ends successfully and a valid patron id has been entered into the system, the system initiates the Enter resource use case. If the Enter resource use case ends successfully, the Determine due date use case is initiated. The Enter resource and Determine due date use-case-pair is executed over and over until the Library staff commits the entry by pressing the enter key. The system then displays a list of valid resources that have been entered along with due dates for each resource. This ends the Check out resource use case.

- Use case format -- LMS

- Exceptional flow of events: If the Validate patron use case does not end with a valid patron id being entered, the system cancels the entire transaction and this use case ends

7

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Exceptional flow of events: if the Enter resource use case does not end with a valid resource Dewey call number having been entered, the system displays an appropriate warning message and continues the Check out resource use case by skipping the Determine due date use case and initiating the next instance of the Enter resource use case.

- Use case format -- LMS

- Postcondition: If the Validate patron use case does not end with a valid patron id being entered, nothing in the entire system has changed when this use case ends. If a valid library patron id was entered, then the Patron object is updated with the Dewey call numbers of the Resource objects that have been checked out along with their due dates. In addition, the Resource objects are updated to have a status

of checked-out along with the library id of the Patron who check it out.

- Use Case: Validate Patron
- Use case: Validate patron
- Precondition: Patron needs some service
- Main flow of events: The use case starts when the system prompts the Library staff for the Patron's library id. The Library staff can either scan in a card or type in an id and then commit the entry by pressing the enter key. The system then checks to see if the id is valid. If it is valid the system displays the library record for Patron, thus ending this use case
- Use case - Validate patron
- Exceptional flow of events: If the Library staff enters an invalid library id, the system cancels the entire transaction and this use case ends
- Exceptional flow of events: If the Library staff enters a library id that has expired then the system cancels the entire transaction and this use case ends
- Exceptional flow of events: If the Patron has too many resources checked out, he or she may not check out any additional resources. A message that tallies and lists the resources currently held by Patron is displayed on the screen and this use case ends
- Use case - Validate patron
- Exceptional flow of events: If the Patron has any library resources that are more than two weeks overdue, then these resources are listed on the screen and this use case ends.

8

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Postcondition: The system retrieves and displays the library record of the Patron. The system's state has not changed.

Architectural design

- Once interactions between the system and its environment have been understood, you use this information for designing the system architecture
- There should be no more than 7 entities in an architectural model

Key points

- OOD is an approach to design so that design components have their own private state and operations
- Objects should have constructor and inspection operations. They provide services to other objects
- Objects may be implemented sequentially or concurrently
- The Unified Modeling Language provides different notations for defining different object models

- Key points
- A range of different models may be produced during an object-oriented design process. These include static and dynamic system models
- Object-oriented design simplifies system evolution

9

Software Engineering Lecture Notes. Dr. David A. Oyemade

4 Design with Reuse

- Building software from reusable components.
- In most engineering disciplines, systems are designed by composing existing components that have been used in other systems
- Software engineering has been more focused on original development but it is now recognised that to achieve better software, more quickly and at lower cost, we need to adopt a design process that is based on systematic reuse

Reuse-based software engineering

- Application system reuse
 - The whole of an application system may be reused either by incorporating it without change into other systems (COTS reuse) or by developing application families
- Component reuse
 - Components of an application from sub-systems to single objects may be reused
- Function reuse
 - Software components that implement a single well-defined function may be reused

Reuse practice

- Application system reuse

10

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Widely practised as software systems are implemented as application families. COTS reuse is becoming increasingly common
- Component reuse
 - Now seen as the key to effective and widespread reuse through component-based software engineering. However, it is still relatively immature
- Function reuse
 - Common in some application domains (e.g. engineering) where domain-specific libraries of reusable functions have been established

Benefits of reuse

- Increased reliability
- Components exercised in working systems
- Reduced process risk
- Less uncertainty in development costs
- Effective use of specialists
- Reuse components instead of people
- Standards compliance
- Embed standards in reusable components
- Accelerated development
- Avoid original development and hence speed-up production

11

Software Engineering Lecture Notes. Dr. David A. Oyemade

Requirements for design with reuse

- It must be possible to find appropriate reusable components
- The re-user of the component must be confident that the components will be reliable and will behave as specified
- The components must be documented so that they can be understood and, where appropriate, modified

Reuse problems

- Increased maintenance costs
- Lack of tool support
- Not-invented-here syndrome
- Maintaining a component library
- Finding and adapting reusable components

Generator-based reuse

- Program generators involve the reuse of standard patterns and algorithms
- These are embedded in the generator and parameterised by user commands. A program is then automatically generated
- Generator-based reuse is possible when domain abstractions and their mapping to executable code can be identified
- A domain specific language is used to compose and control these abstractions

Types of program generator

- Application generators for business data processing
- Parser and lexical analyser generators for language processing

- Code generators in CASE tools

12

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Generator-based reuse is very cost-effective but its applicability is limited to a relatively small number of application domains
- It is easier for end-users to develop programs using generators compared to other component-based approaches to reuse

Reuse through program generation

Component-based development

- Component-based software engineering (CBSE) is an approach to software development that relies on reuse
- It emerged from the failure of object-oriented development to support effective reuse. Single object classes are too detailed and specific
- Components are more abstract than object classes and can be considered to be stand-alone service providers

Components

- Components provide a service without regard to where the component is executing or its programming language

13

Software Engineering Lecture Notes. Dr. David A. Oyemade

- A component is an independent executable entity that can be made up of one or more executable objects
- The component interface is published and all interactions are through the published interface
- Components can range in size from simple functions to entire application systems

Component interfaces

- Provides interface
- Defines the services that are provided by the component to other components
- Requires interface
- Defines the services that specifies what services must be made available for the component to execute as specified

Printing services component

14

Software Engineering Lecture Notes. Dr. David A. Oyemade

Component abstractions

- Functional abstraction
 - The component implements a single function such as a mathematical function
- Casual groupings
 - The component is a collection of loosely related entities that might be data declarations, functions, etc.
- Data abstractions
 - The component represents a data abstraction or class in an object-oriented language
- Cluster abstractions
 - The component is a group of related classes that work together
- System abstraction
 - The component is an entire self-contained system

CBSE processes

- Component-based development can be integrated into a standard software process by incorporating a reuse activity in the process

15

Software Engineering Lecture Notes. Dr. David A. Oyemade

- However, in reuse-driven development, the system requirements are modified to reflect the components that are available
- CBSE usually involves a prototyping or an incremental development process with components being 'glued together' using a scripting language

An opportunistic reuse process

Development with reuse

CBSE problems

- Component incompatibilities may mean that cost and schedule savings are less than expected
- Finding and understanding components
- Managing evolution as requirements change in situations where it may be impossible to change the system components

16

Software Engineering Lecture Notes. Dr. David A. Oyemade

Application frameworks

- Frameworks are a sub-system design made up of a collection of abstract and concrete classes and the interfaces between them
- The sub-system is implemented by adding components to fill in parts of the

design and by instantiating the abstract classes in the framework

- Frameworks are moderately large entities that can be reused
- Framework classes
- System infrastructure frameworks
 - Support the development of system infrastructures such as communications, user interfaces and compilers
- Middleware integration frameworks
 - Standards and classes that support component communication and information exchange
- Enterprise application frameworks
 - Support the development of specific types of application such as telecommunications or financial systems

Extending frameworks

- Frameworks are generic and are extended to create a more specific application or sub-system
- Extending the framework involves
 - Adding concrete classes that inherit operations from abstract classes in the framework
 - Adding methods that are called in response to events that are recognised by the framework
- Problem with frameworks is their complexity and the time it takes to use them effectively
- Model-view controller

17

Software Engineering Lecture Notes. Dr. David A. Oyemade

- System infrastructure framework for GUI design
- Allows for multiple presentations of an object and separate interactions with these presentations

MVC framework involves the instantiation of a number of Model-view controller

COTS product reuse

- COTS - Commercial Off-The-Shelf systems
- COTS systems are usually complete application systems that offer an API (Application Programming Interface)
- Building large systems by integrating COTS systems is now a viable development strategy for some types of system such as E-commerce systems
- COTS system integration problems

- Lack of control over functionality and performance
- COTS systems may be less effective than they appear
- Problems with COTS system inter-operability
- Different COTS systems may make different assumptions that means integration is difficult
- No control over system evolution
- COTS vendors not system users control evolution
- Support from COTS vendors
- COTS vendors may not offer support over the lifetime of the product

Component development for reuse

- Components for reuse may be specially constructed by generalising existing components

18

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Component reusability
- Should reflect stable domain abstractions
- Should hide state representation
- Should be as independent as possible
- Should publish exceptions through the component interface

There is a trade-off between reusability and usability.

- The more general the interface, the greater the reusability but it is then more complex and hence less usable

Reusable components

- The development cost of reusable components is higher than the cost of specific equivalents. This extra reusability enhancement cost should be an organization rather than a project cost
- Generic components may be less space-efficient and may have longer execution times than their specific equivalents

Reusability enhancement

- Name generalisation
- Names in a component may be modified so that they are not a direct reflection of a specific application entity
- Operation generalisation
- Operations may be added to provide extra functionality and application specific operations may be removed

- Exception generalisation
- Application specific exceptions are removed and exception management added to increase the robustness of the component

19

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Component certification
- Component is certified as reusable

Reusability enhancement process

Application families

- An application family or product line is a related set of applications that has a common, domain-specific architecture
- The common core of the application family is reused each time a new application is required
- Each specific application is specialised in some way

Application family specialisation

- Platform specialisation
- Different versions of the application are developed for different platforms
- Configuration specialisation
- Different versions of the application are created to handle different peripheral devices
- Functional specialisation
- Different versions of the application are created for customers with different requirements

Inventory management systems

20

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Resource database
- Maintains details of the things that are being managed

Application family architectures

- Architectures must be structured in such a way as to separate different sub-systems and to allow them to be modified
- The architecture should also separate entities and their descriptions and the higher levels in the system access entities through descriptions rather than directly

Library system

- The resources being managed are the books in the library
- Additional domain-specific functionality (issue, borrow, etc.) must be added for

this application

Family member development

- Elicit stakeholder requirements
- Use existing family member as a prototype
- Choose closest-fit family member
- Find the family member that best meets the requirements
- Re-negotiate requirements
- Adapt requirements as necessary to capabilities of the software

21

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Adapt existing system
- Develop new modules and make changes for family member
- Deliver new family member
- Document key features for further member development

Design patterns

- A design pattern is a way of reusing abstract knowledge about a problem and its solution
- A pattern is a description of the problem and the essence of its solution
- It should be sufficiently abstract to be reused in different settings
- Patterns often rely on object characteristics such as inheritance and polymorphism
- Pattern elements
- Name
- A meaningful pattern identifier
- Problem description
- Solution description
- Not a concrete design but a template for a design solution that can be instantiated in different ways
- Consequences
- The results and trade-offs of applying the pattern
- Multiple displays
- Key points

22

Software Engineering Lecture Notes. Dr. David A. Oyemade

- Design with reuse involves designing software around good design and existing components

- Advantages are lower costs, faster software development and lower risks
- Component-based software engineering relies on black-box components with defined requires and provides interfaces
- COTS product reuse is concerned with the reuse of large, off-the-shelf systems

Key points

- Software components for reuse should be independent, should reflect stable domain abstractions and should provide access to state through interface operations
- Application families are related applications developed around a common core
- Design patterns are high-level abstractions that document successful design solutions

23

Software Engineering Lecture Notes. Dr. David A. Oyemade

Assignment: Model your final year project using object oriented analysis and design (OOAD). Use CASE tools for the modeling and an object oriented programming language such as Java, C# or C++ for the implementation.

(30/6/2016).

24